

The L_{Var} language (Project 2)

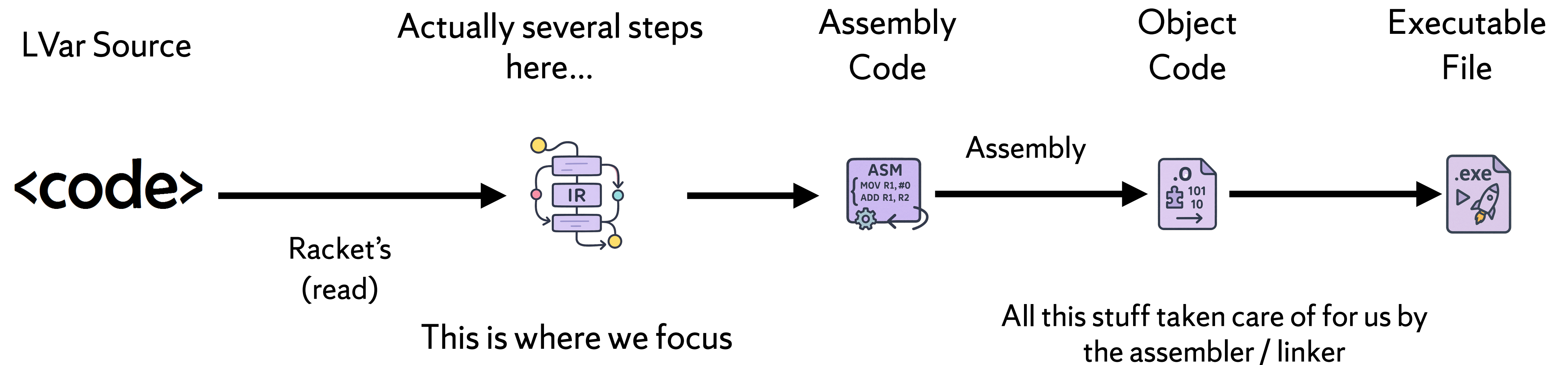
CIS531 — Fall 2025, Syracuse

Kristopher Micinski

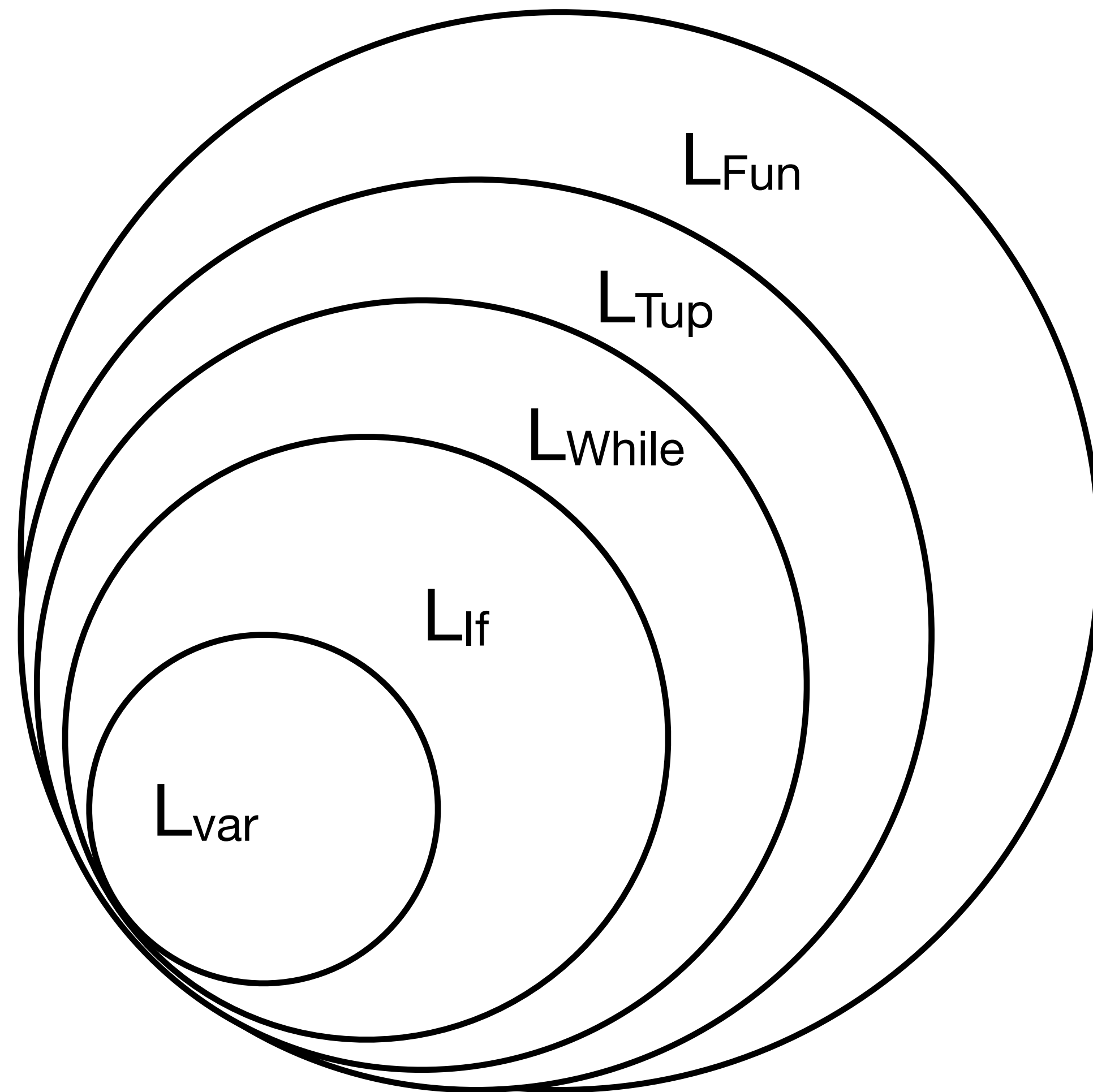


Project 2 in CIS352

- * This week we'll dive into Project 2 in CIS352
- * You'll write a compiler for LVar, a language that extends R0 to several other forms
- * You'll also compile LVar to x86_64 assembly and verify it works
- * Project due in roughly 14 days



The LVar language



We cover LVar, the first language in the stack

Order of language the book presents:

- **LVar — integer +/-, straight-line variables**
- LIf — branching control
- LWhile — loops
- LTup — heap-allocated data, GC
- LFun — functions
- Some more...

The LVar (R1) language (Grammar)

$$\begin{array}{lcl} \text{exp} ::= & \text{int} & | \text{ (read) } & | \text{ (- exp) } & | \text{ (+ exp exp) } \\ & & | \text{ var} & | \text{ (let ([var exp]) exp) } \end{array}$$
$$R1 ::= \text{ (program exp) }$$

Notice that compared to R0, R1 adds **variables** and **let-binding**
Handling naming, shadowing, etc. will present novel challenges for us to solve

Livcoding: let's write an interpreter for LVar / R1

In-class demo...

The x86 language

This chapter covers a specific subset of x86_64:

- Only 64-bit integers
- Limited addressing modes
- Only **eight** instructions (plus labels)
- Whole program is just consecutive instructions in one function: main

```
reg    ::=  rsp | rbp | rax | rbx | rcx | rdx | rsi | rdi |  
          r8 | r9 | r10 | r11 | r12 | r13 | r14 | r15  
arg    ::=  $int | %reg | int(%reg)  
instr  ::=  addq arg, arg | subq arg, arg | negq arg | movq arg, arg |  
          callq label | pushq arg | popq arg | retq | label: instr  
prog   ::=  .globl main  
          main: instr+
```

Figure 2.3: A subset of the x86 assembly language (AT&T syntax).

x86: what you need to know for P1

- Assembly code consists of **instruction sequences**
 - Grouped into “functions” (procedures)
- Each instruction does a very simple task (add, multiply, jump)
- There are a limited number of variables (registers)
 - x86-64 has 16 of these! 2 hold pointers to stack (rsp/rbp)
- If you need more memory (e.g., for storing an array), must store in stack / heap / etc...
- We avoid “functions” for now—mostly

x86_64 is huge, but you only need to know a little bit...

- Ugly instruction set: decades of development and extensions
 - x86_64 is the exemplar “CISC” ISA
- In practice, you will deal with a very small subset of x86_64
 - Nearly half the instruction space (by encoding) is AVX/SIMD!
- For a while, we will focus on compiling only a single function

This program should print out 500 to stdout

(program 500)

```
clang -arch x86_64 -o ex1 ex1.s runtime.c
```

```
.text
.globl main
.extern print_int64

main:
    push    %rbp
    mov     %rsp, %rbp
    movq    $500, %rdi
    call    print_int64
    leave
    ret
```

(On Mac: make sure to prefix all symbols with _, i.e., `_main` and `_print_int64`)

```
.text
.globl main
.extern print_int64
```

main:

```
    push    %rbp
    mov     %rsp, %rbp
    movq    $42, %rdi
    call    print_int64
    leave
    ret
```

Commands starting with dots (.) are directives that tell the assembler how to lay out your program


```
    .text
    .globl main
    .extern print_int64

main:
    push    %rbp
    mov     %rsp, %rbp
    movq    $42, %rdi
    call    print_int64
    leave
    ret
```

The OS loader **assumes** your binary will have a function named main (Linux ELF) or (on Mac) `_main`

While Linux / Mac are very close, there is one very key difference: labels in the Mach ABI use **underscores**

So printf is `_printf`, main is `_main`, etc.

```

    .text
    .globl main
    .extern print_int64

main:
    push    %rbp
    mov     %rsp, %rbp
    movq    $42, %rdi
    call    print_int64
    leave
    ret

```

Briefly, here's what the snippet does:

- **Function prelude:**

- Each function needs to set things up
- In this case, we push the “base pointer” %rbp
- Then, move the “stack pointer” into %rbp
 - (Source then target in GAS Assembler)

- **Function body:**

- We move the literal value 42 (decimal) into %rdi
 - %rdi is a general-purpose register—we'll cover this soon
 - First argument to each function is passed in %rdi
- Next, we call print_int64 (arg in %rdi)

- **Function epilogue:**

- Each function needs to clean things up
- leave pops %rbp, return exits from the function

Cross-compiling on a Mac (M1/2/3/...)

- ✱ Remember, if on a Mac **prefix all symbols with _**
- ✱ We use clang's *cross-compiler* to build a Mach-O x86_64 object file
- ✱ Then, we use clang to link the object (.o) file into a Mach-O executable
- ✱ Finally, we run it: Rosetta loads it and translates it on-the-fly to run on ARM!

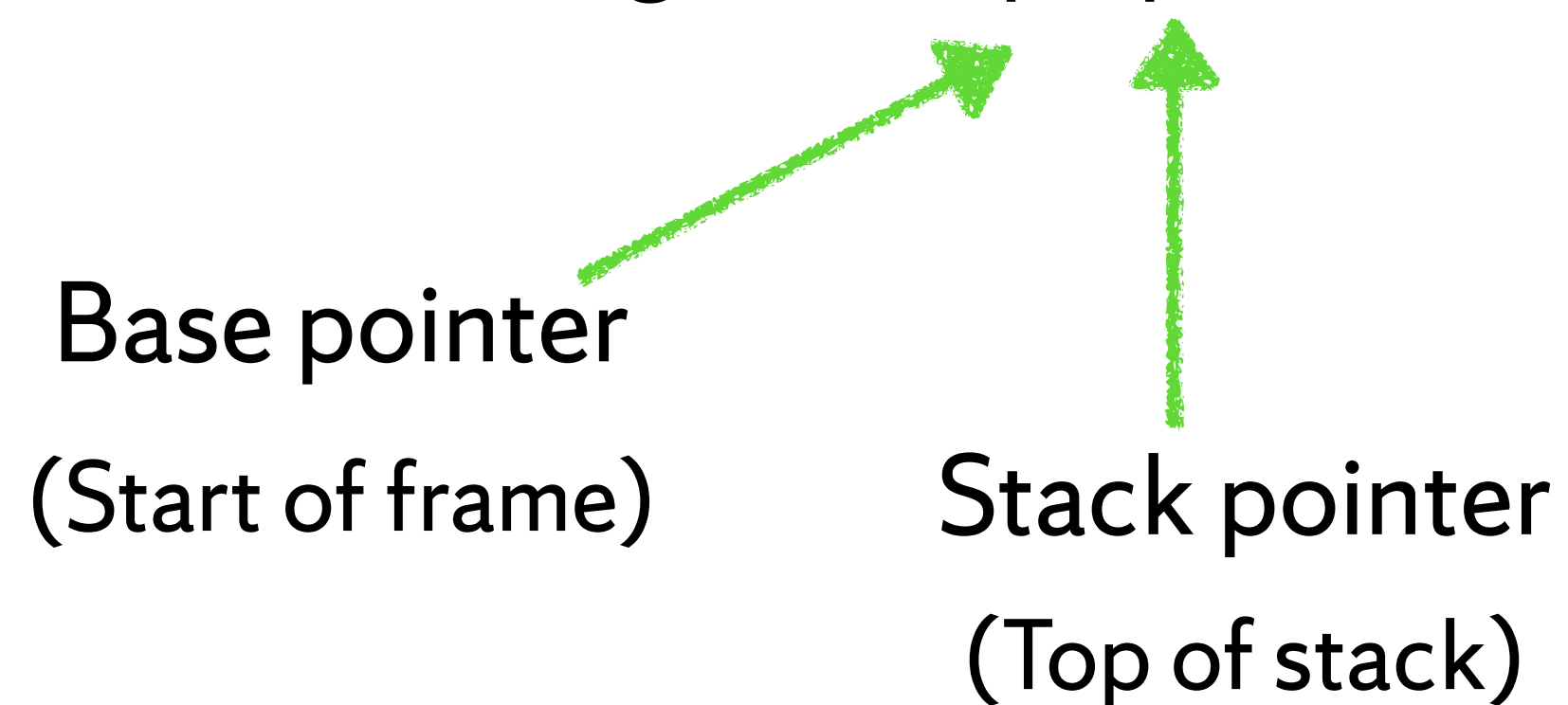
```
kkmicins@laptop ∅ → clang -target x86_64-apple-darwin -c ex0.s
kkmicins@laptop ∅ → clang -target x86_64-apple-darwin ex0.o -o prog
kkmicins@laptop ∅ → ./prog
Hello, world!
```

Registers: very fast, on-chip data

Originally, 8-bit registers: al, bl, cl, dl

Historically, x86 architectures only had **four** 16-bit general purpose registers: ax, bx, cx, dx

Also other registers: bp, sp, di, si



Registers: very fast, on-chip data

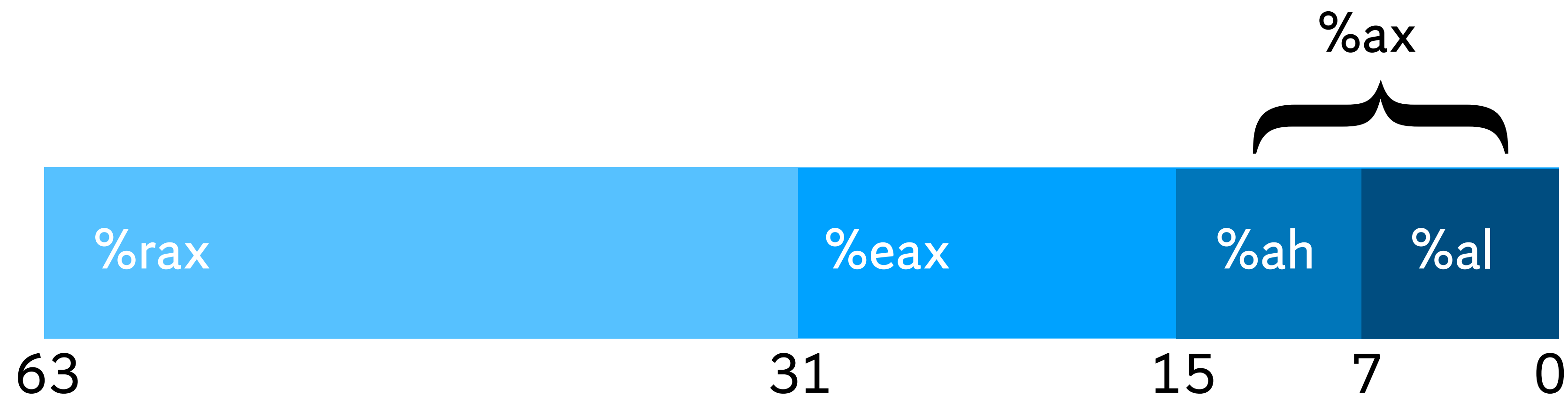
- x86-64: 16 “general purpose” registers: rax, rbx, rcx, rdx, rsp, rbp, rsi, rdi, R8-R15
- Also some special registers you can’t modify directly: instruction pointer, flags
- Registers are a **precious commodity**: you want to store temporaries in registers
 - Historically: ensuring something certain values kept in registers was a reason to write manually-tuned Assembly code (not as common in modern times)

Preview: register allocation (towards the end of the course...)

- You will eventually **run out** of registers, at which point you **have** to “spill” to the stack
 - Stack stores local values: medium-sized chunk of memory that you can push/pop
 - Push on function entry, pop on function exit—local values disappear once function done

Smaller registers are *nested* inside bigger ones

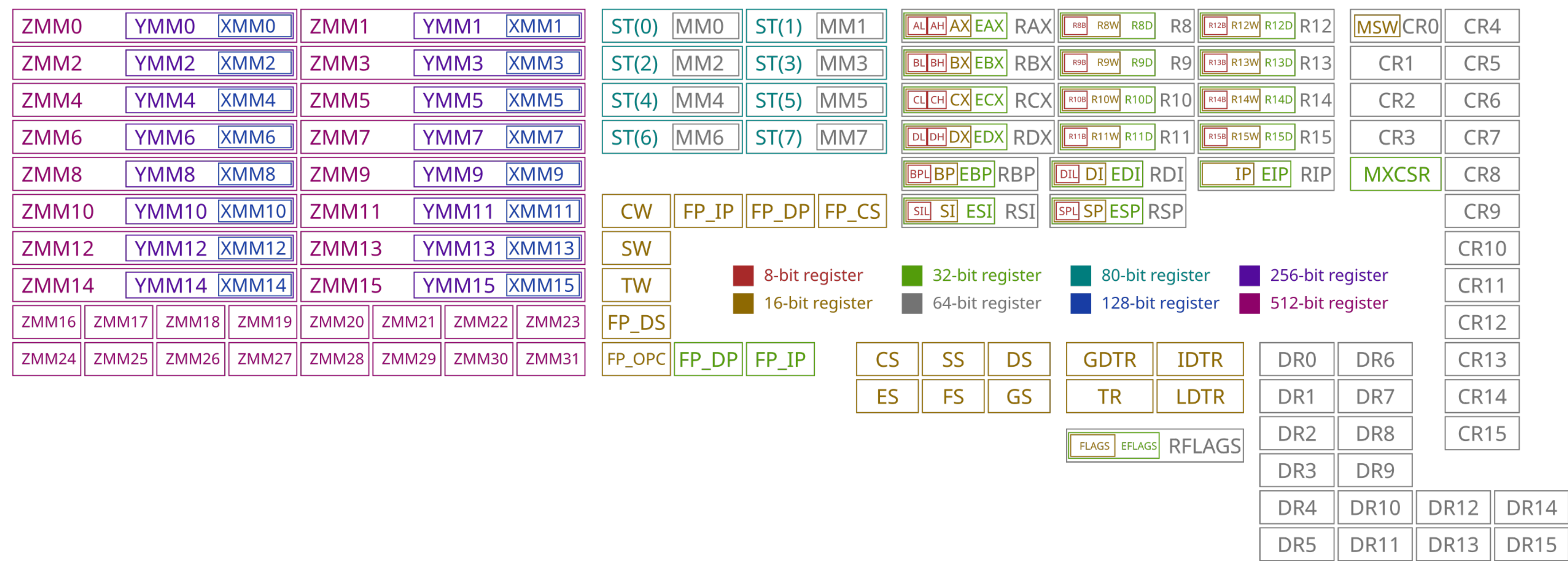
- %rax is one of the most common general-purpose registers
- But, if you use %eax, you're **really** using the **lower 32 bits** of %rax
- In this class, we'll make it easy: everything is 64 bits!



Registers in x86_64

- Today we mostly have 64-bit ISAs
- 32-bit registers still used (e.g., int in C++ on my machine)
- 16-bit basically gone, 8-bit totally gone
 - Some code out there that still works with the 32/16/8 bit registers, though
- Assembly instructions have **suffixes**: denote which bit-width they're working on
 - E.g., movq says "move into a **quadword** (8-bytes)"
 - "Word" is ambiguous ("machine word" is the native register size)
 - movl says "move into a **long** (4-bytes)"
 - We will try to keep things simple by working with 64-bit values values!

x86_64 register map



https://commons.wikimedia.org/wiki/File:Table_of_x86_Registers_svg.svg

Exercise:

```
.text
.globl main
.extern print_int64

main:
    push    %rbp
    mov     %rsp, %rbp
    ...
    # put something into %rdi
    call    print_int64
    leave
    ret
```

Translate the following into pseudocode:

$x = 40$

$y = 20$

$z = 2 + x * y$

Using the registers %rax, %rbx, %rcx, %rdx (if needed...)

And the instructions:

add %src, %dst # %dst = %src + %dst

imul %src, %dst # %dst = %src * %dst

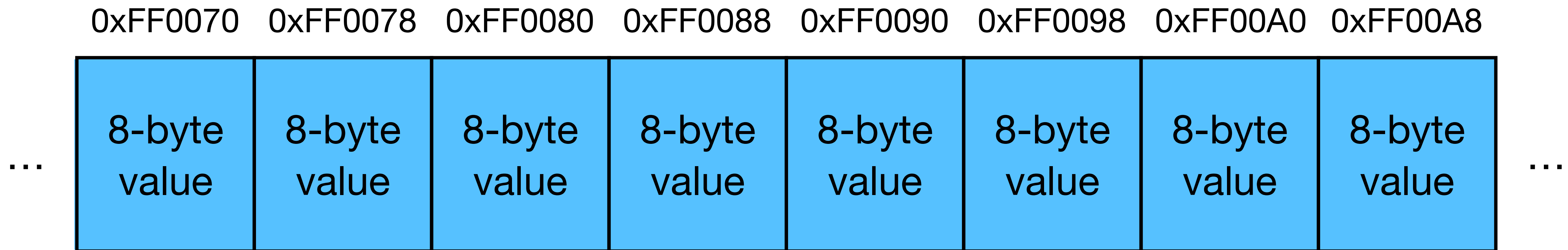
mov %src, %dst # put result into %rdi

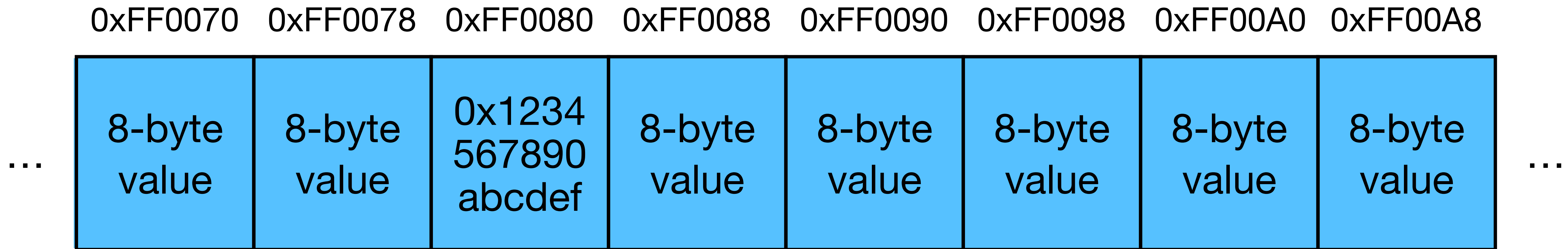
Ultimately, we will sometimes run out of registers...

- If we only have N registers, we can only write programs that use $N \cdot 8$ bytes!
- Instead, also keep data in **memory**
 - When we run out of registers, use RAM
- But ultimately, most instructions **require** at least one operand to be a register
 - Solution: registers track “current” data, contents can be “spilled to” the stack
- Memory loads / stores **dominate the execution time of most modern apps**
 - Most chips will have a hierarchy of caches
 - Recently-used data will generally be in the cache—not full trip to RAM

Memory: a big sea of bytes

From the POV of an assembly, we can think of memory as being a “giant array”





We can **load** from memory and **store** to memory using mov

We need a **pointer** to some value stored in a register...

For example, let's say we wanted to load 0xFF0080 into %rcx,
assuming that %rax = 0xFF0080

```
// in C: %rcx = *%rax  
movq (%rax), %rcx
```

“Move the value **at address** %rax into register %rbx”

Opcode name

Destination

mov (%rax), %rbx

Source

%rax

0xffffffff00000000

0xffffffff00000008

0xaf23c8a223356ac

%rbx

0x1234123412341234

0xffffffff00000000

0xdeadbeefdeadbeef

“Move the value **at address** %rax into register %rbx”

Opcode name

Destination

mov (%rax), %rbx

Source

%rax

0xffffffff00000000

0xffffffff00000008

0xaf23c8a223356ac

%rbx

0xdeadbeefdeadbeef

0xffffffff00000000

0xdeadbeefdeadbeef



“Move the value **at address** %rax+8 into register %rbx”

Opcode name

Destination

mov 8(%rax), %rbx

Source

%rax

0xffffffff00000000

0xffffffff00000008

0xaf23c8a223356ac

%rbx

0xaf23c8a223356ac

0xffffffff00000000

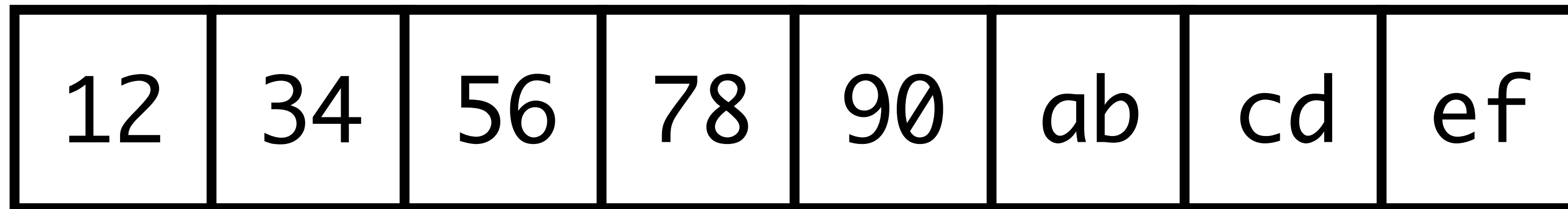
0xdeadbeefdeadbeef



Endianness

- x86 is a **little-endian** architecture
- If an n -byte value is stored at addresses a to $a+(n-1)$ in memory, byte a will hold the least significant byte
- Note: values ***within*** bytes *do not* change—only the **byte order** changes
- How would 0x1234567890abcdef be stored in memory on a big/little-endian machine, starting at address 0xAA00?

To represent 0x1234567890abcdef



Most Significant Byte

Least Significant Byte

The only instructions you need to know for P2

```
reg ::= rsp | rbp | rax | rbx | rcx | rdx | rsi | rdi |  
       r8 | r9 | r10 | r11 | r12 | r13 | r14 | r15  
arg  ::= $int | %reg | int(%reg)  
instr ::= addq arg, arg | subq arg, arg | negq arg | movq arg, arg |  
          callq label | pushq arg | popq arg | retq | label: instr  
prog ::= .globl main  
        main: instr+
```

Figure 2.3: A subset of the x86 assembly language (AT&T syntax).

- Arithmetic: **addq**, **subq**, **negq** (destination last, source first)
- Data movement: **movq**, **pushq**, **popq**
- Function call / return: **callq** , **retq**

Boilerplate code for P2

$$exp ::= int \mid (read) \mid (- exp) \mid (+ exp exp) \\ \mid var \mid (let ([var exp]) exp)$$

$R1 ::= (program\ exp)$

```
.text
.globl main
.extern print_int64
```

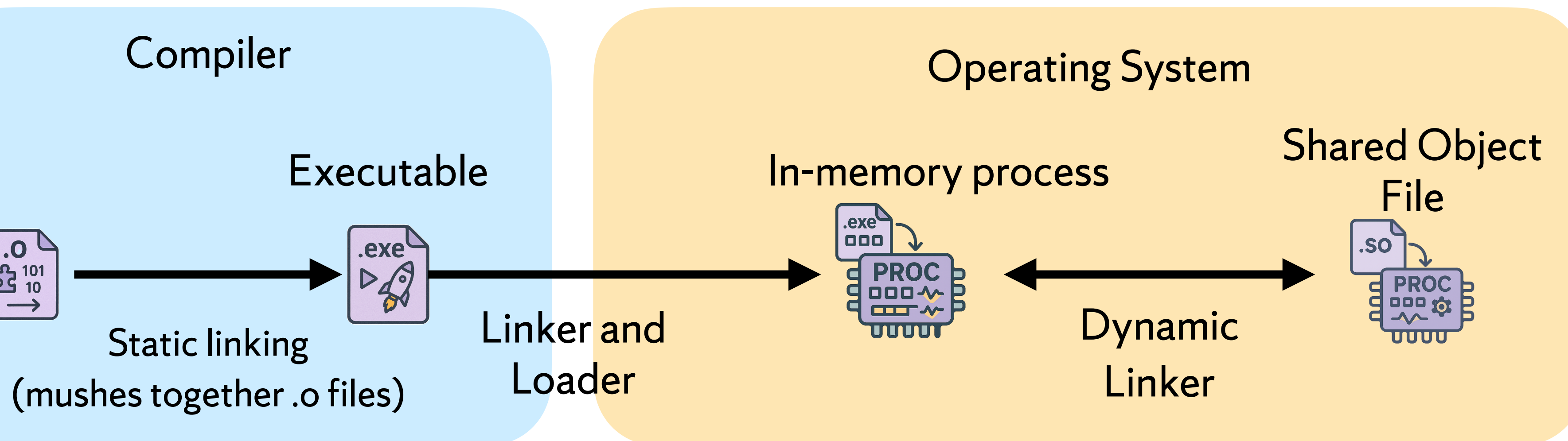
```
main:
    push    %rbp
    mov     %rsp, %rbp
    <GENERATED CODE HERE>
    # Leave return value in %rdi
    call    print_int64
    leave
    ret
```

The crux of our effort will be to translate expressions into assembly language code that implements desired behavior

Leave result of exp in register `%rdi`

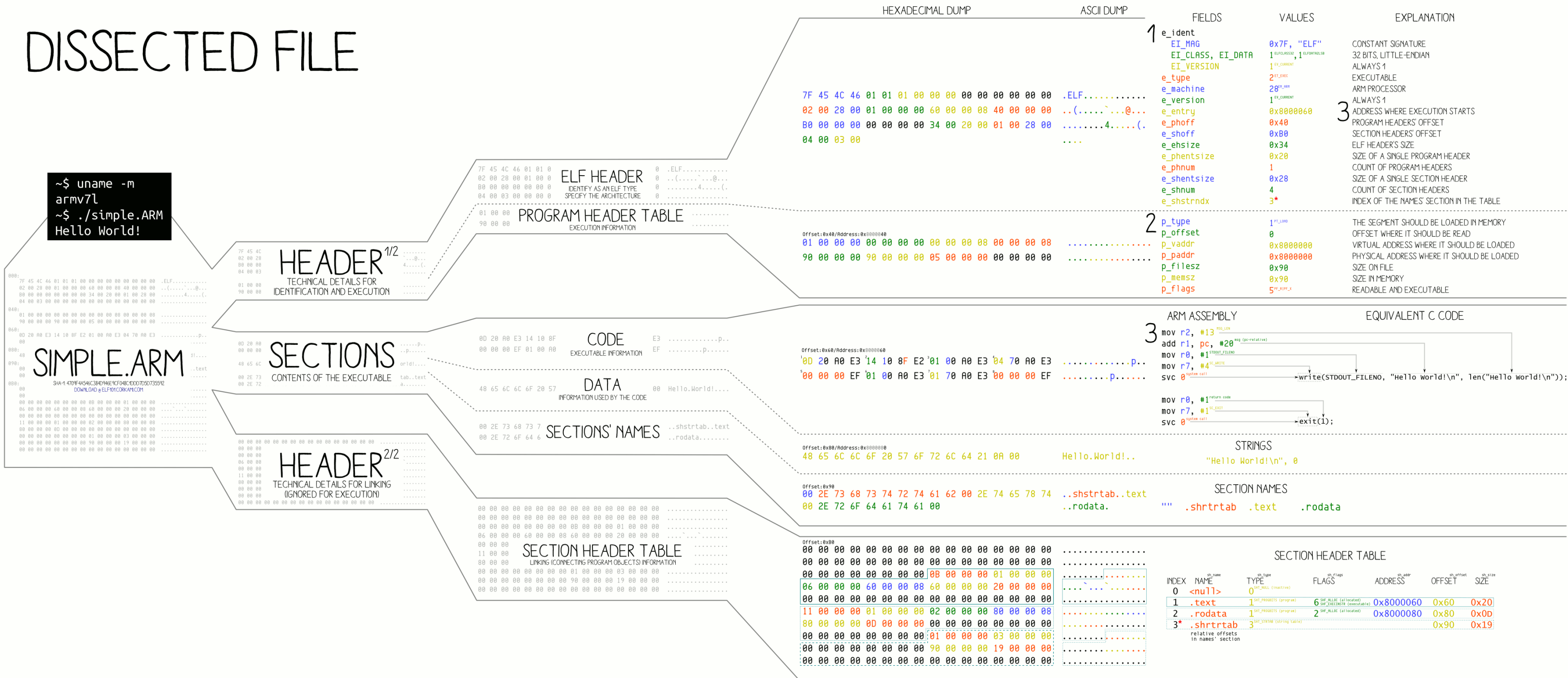
What happens *after* the executable is generated

- The executable is written in a very specific format which the OS understands
 - Examples: Windows Portable Executables (PE .exes), Linux ELF, Mac Mach-O, other lesser-known
- The OS typically organizes process memory into **segments**
- The loader (roughly: what happens after `execve`) knows how to populate the processes memory from the .exe file. The loader and dynamic linker collaborate to load the .exe into memory
- At runtime, some calls to externally-defined functions may need to be resolved by the dynamic linker—this is accomplished by some technical systems tricks which I will not explain in detail now





DISSECTED FILE



LOADING PROCESS

1 HEADER

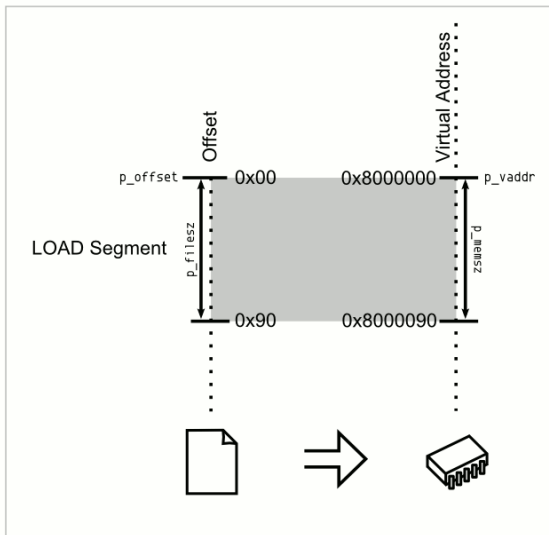
THE ELF HEADER IS PARSED
THE PROGRAM HEADER IS PARSED
(SECTIONS ARE NOT USED)

2 MAPPING

THE FILE IS MAPPED IN MEMORY
ACCORDING TO ITS SEGMENT(S)

3 EXECUTION

ENTRY IS CALLED
SYSCALLS¹⁰⁰ ARE ACCESSED VIA:
- SYSCALL NUMBER IN THE R7 REGISTER¹⁰¹
- CALLING INSTRUCTION SVC¹⁰²



TRIVIA

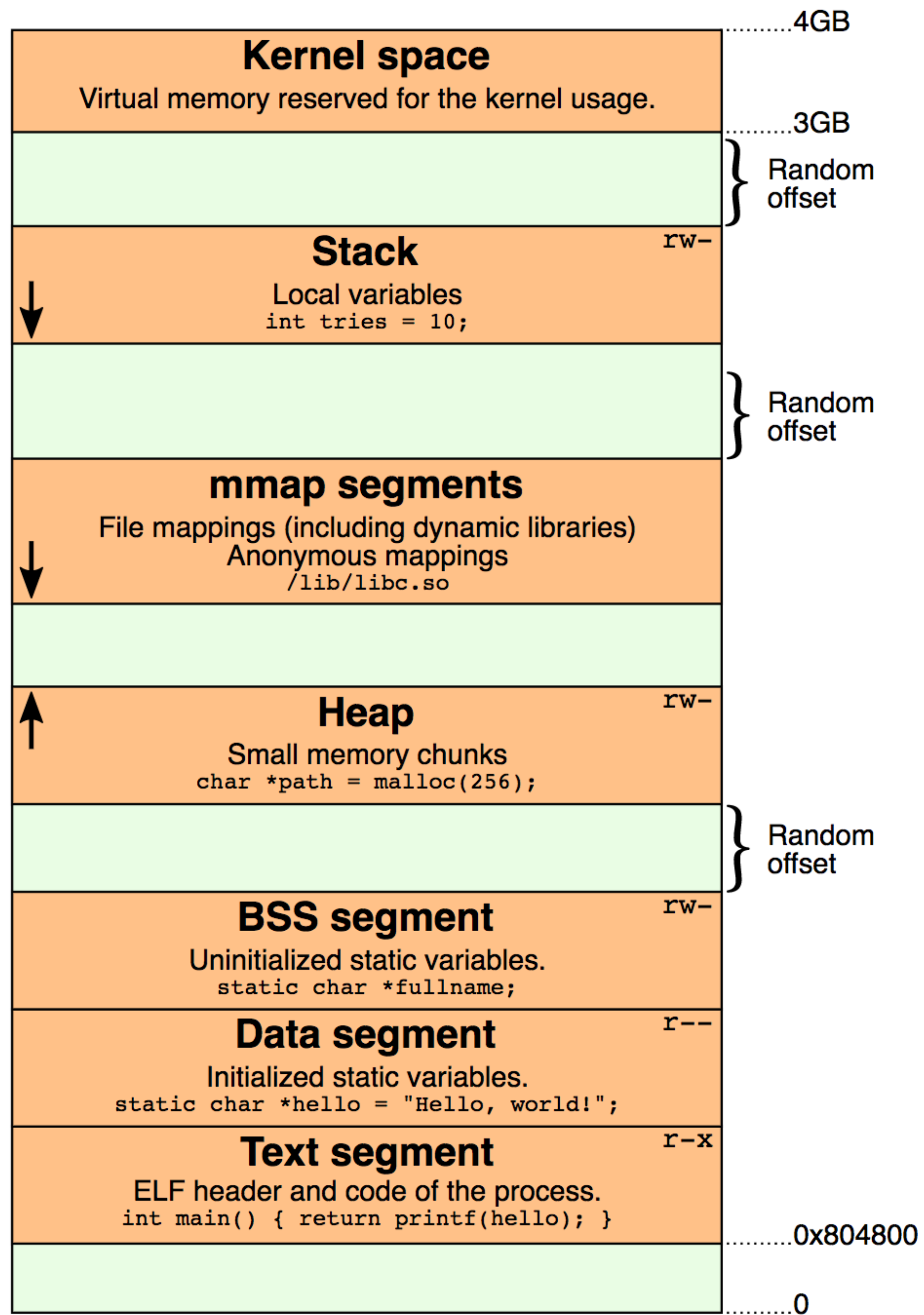
THE ELF WAS FIRST SPECIFIED BY U.S.'L' AND U.I.
FOR UNIX SYSTEM V, IN 1989

THE ELF IS USED, AMONG OTHERS, IN:

- LINUX, ANDROID, *BSD, SOLARIS, BEOS
- PSP, PLAYSTATION 2-4, DREAMCAST, GAMECUBE, WII
- VARIOUS OSES MADE BY SAMSUNG, ERICSSON, NOKIA,
- MICROCONTROLLERS FROM ATMEL, TEXAS INSTRUMENTS

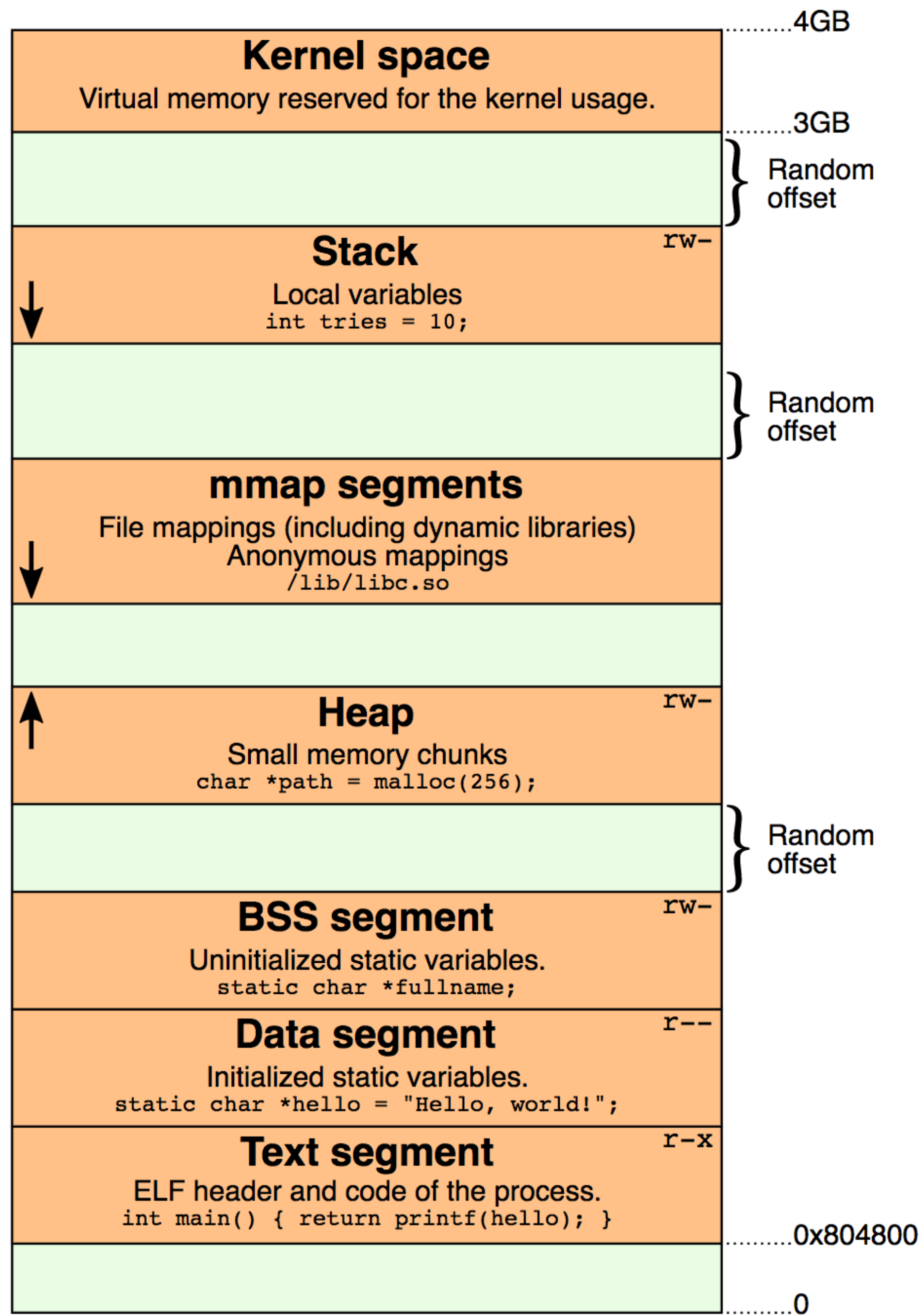
THIS IS THE WHOLE FILE, HOWEVER, MOST ELF FILES CONTAIN MANY MORE ELEMENTS
EXPLANATIONS ARE SIMPLIFIED, FOR CONCISENESS.





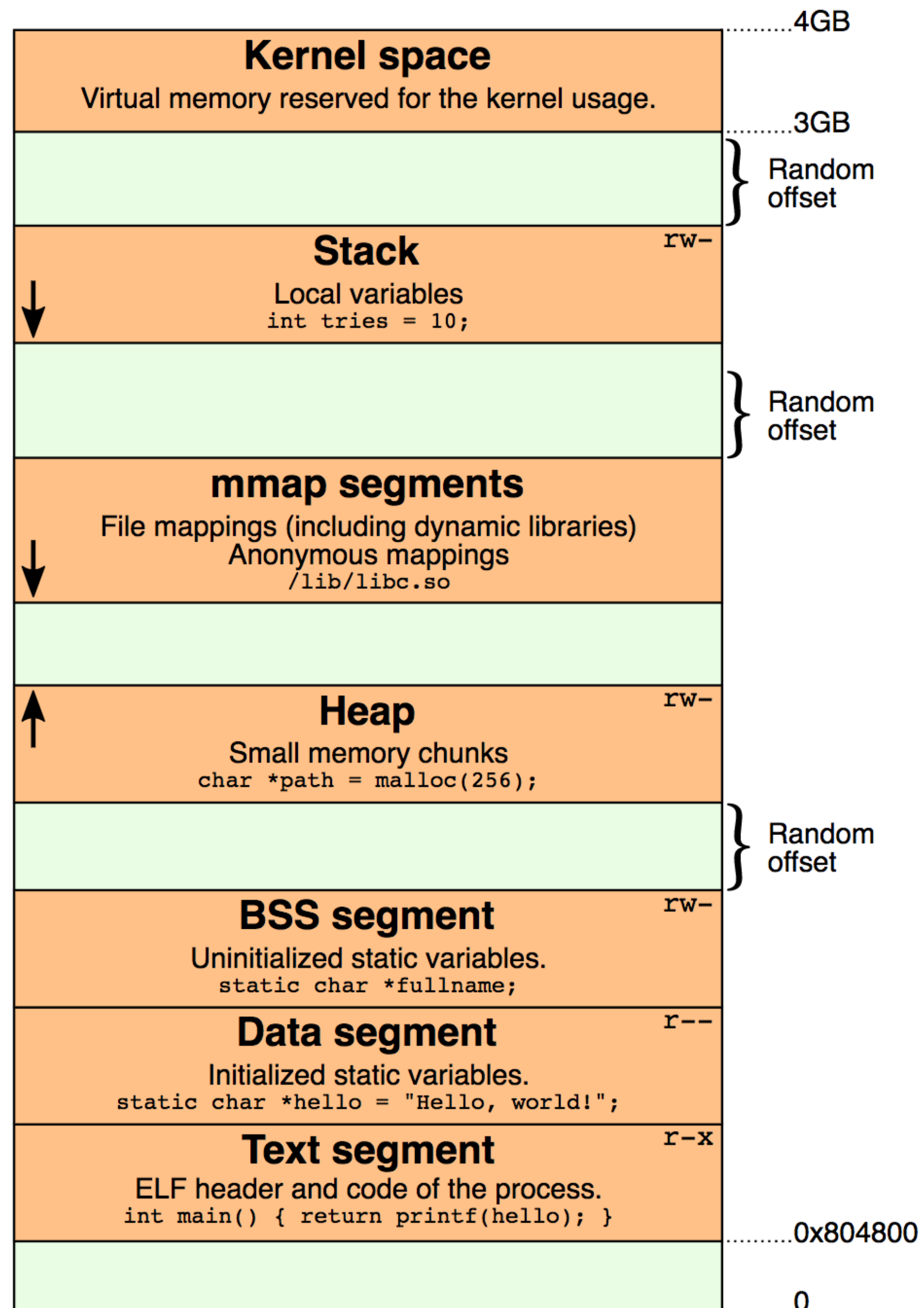
Kernel memory

Your OS uses it



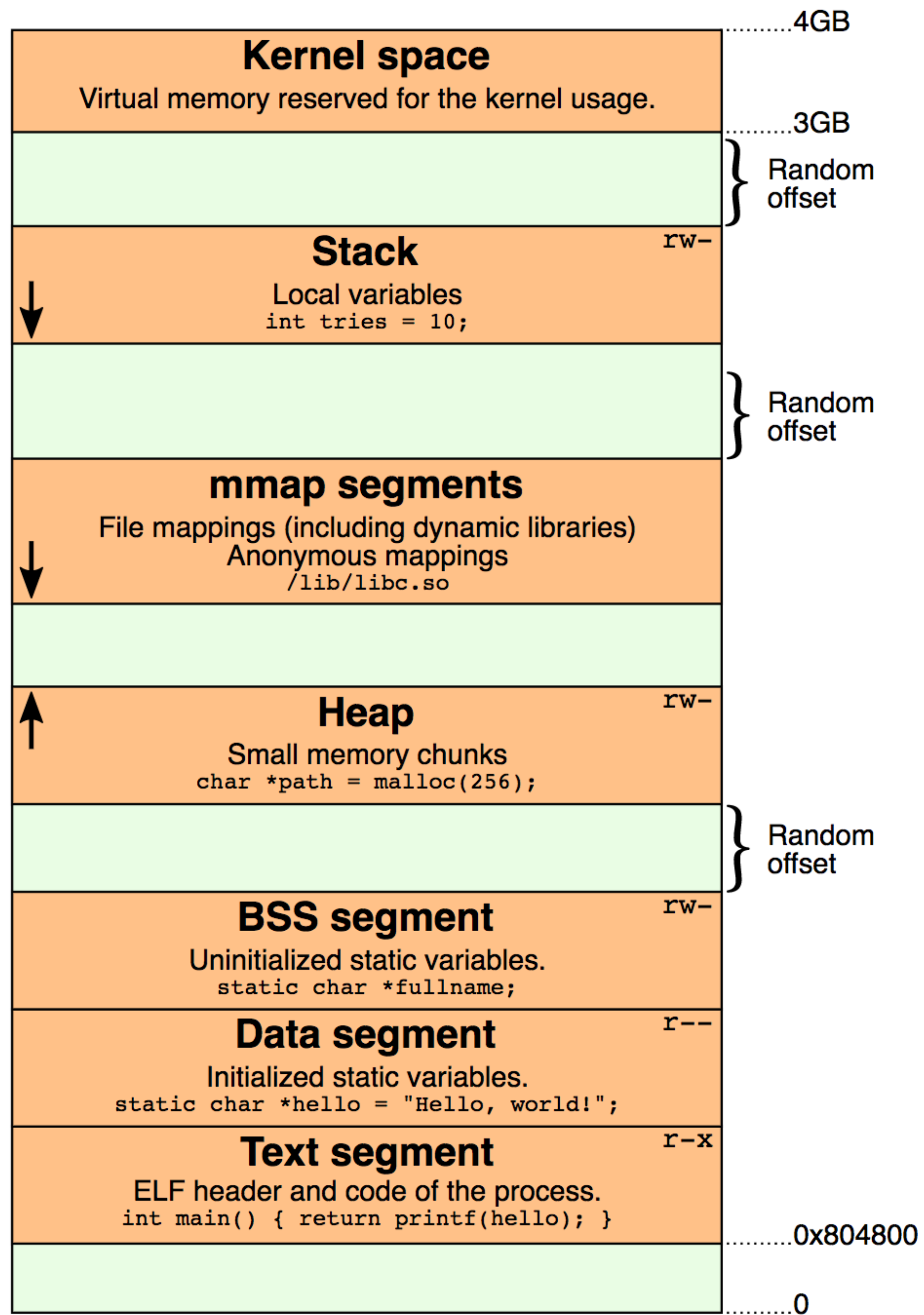
Stack: push / pop

Very important:
The stack grows **down**



mmap segments

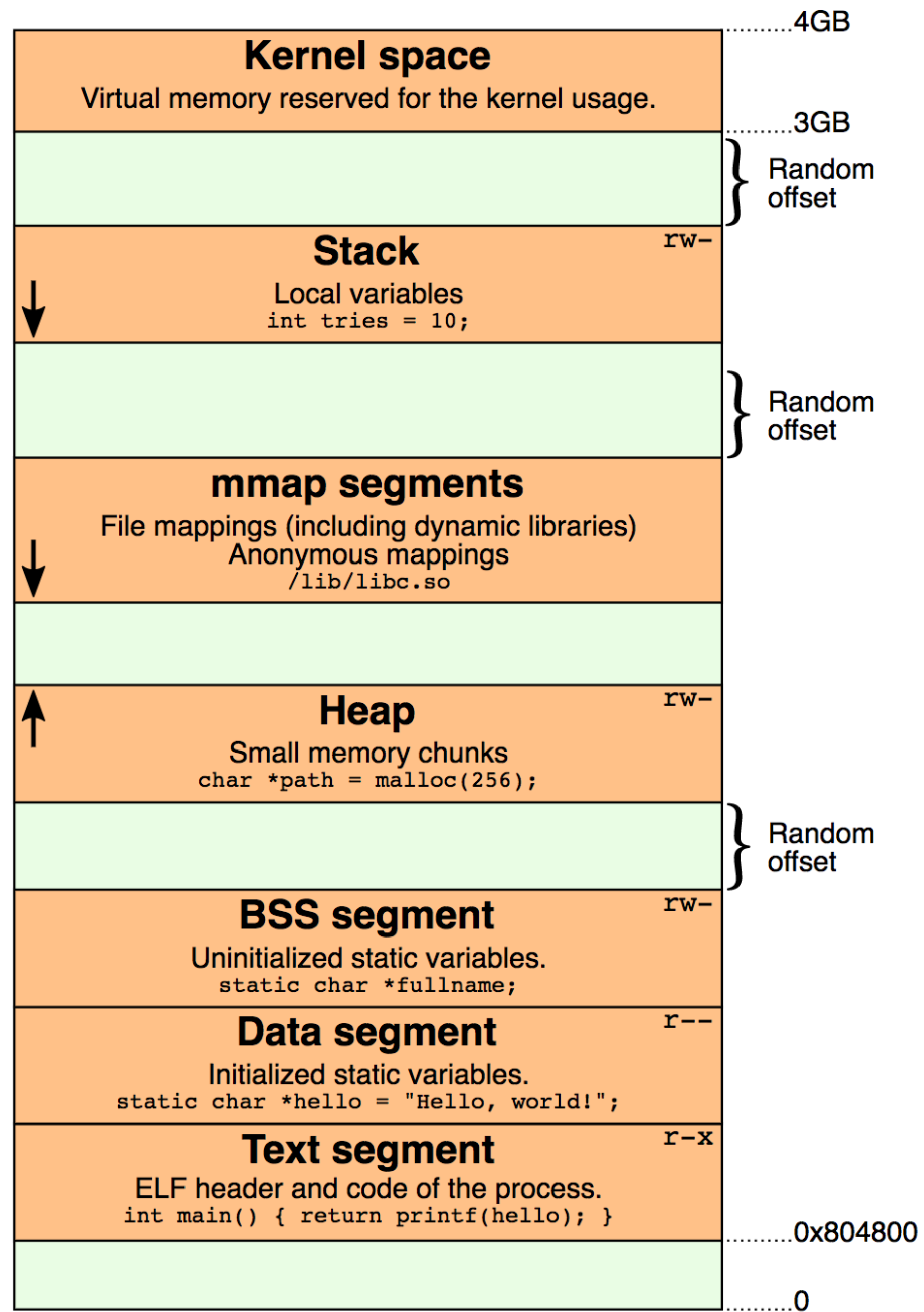
Allows you to **map** a file
to memory



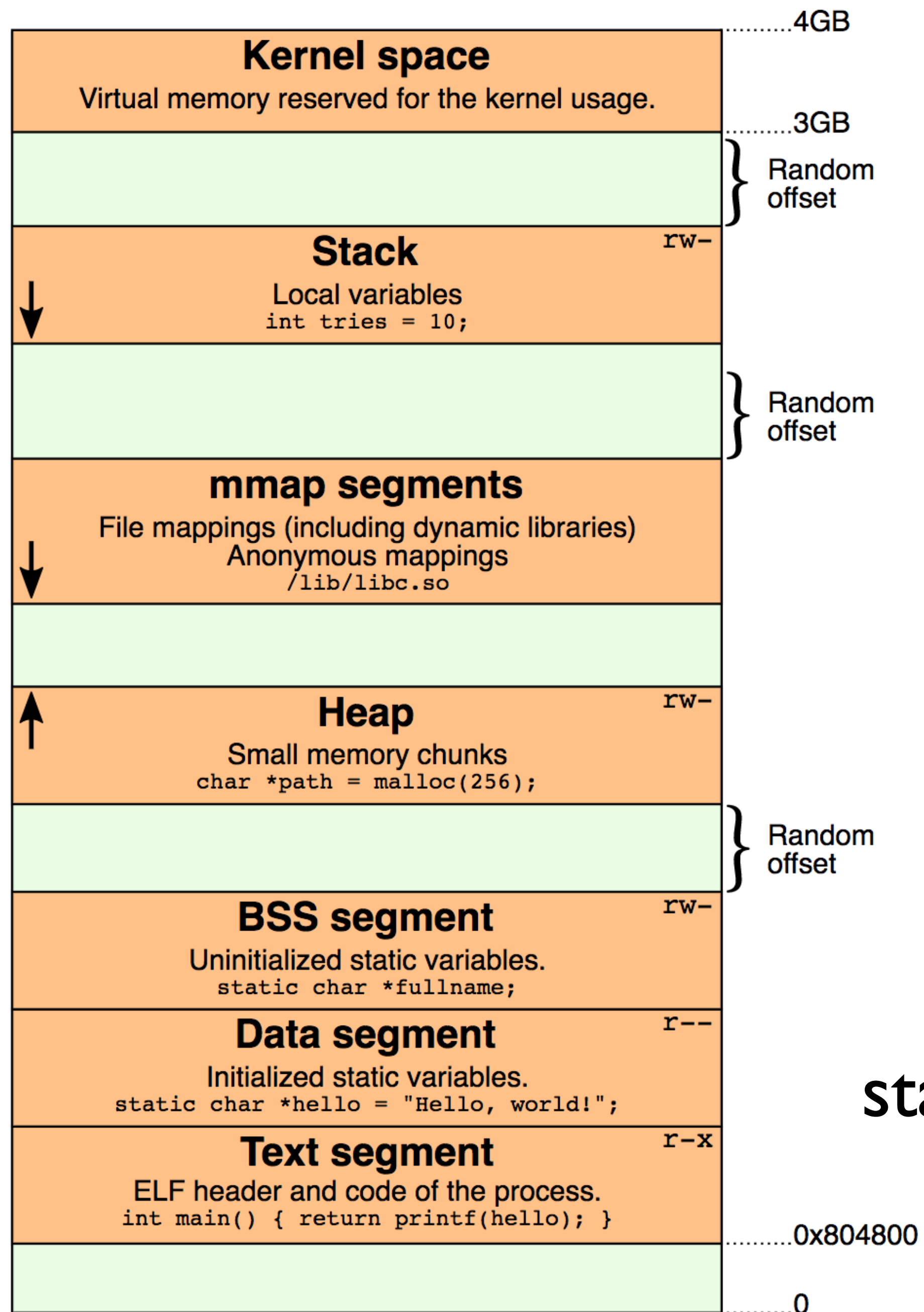
Heap: dynamic allocation

C++: New / delete

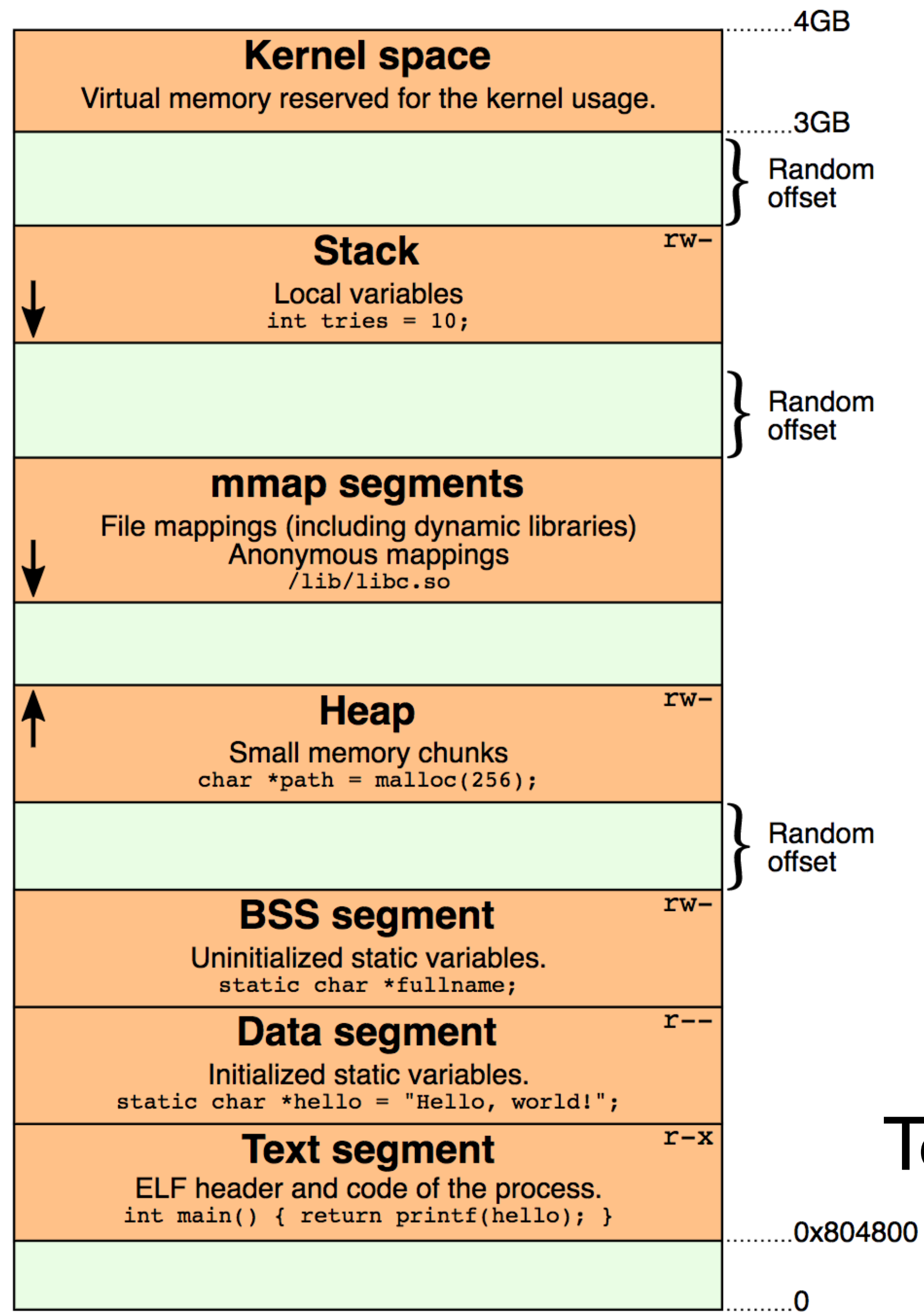
C: Malloc / free



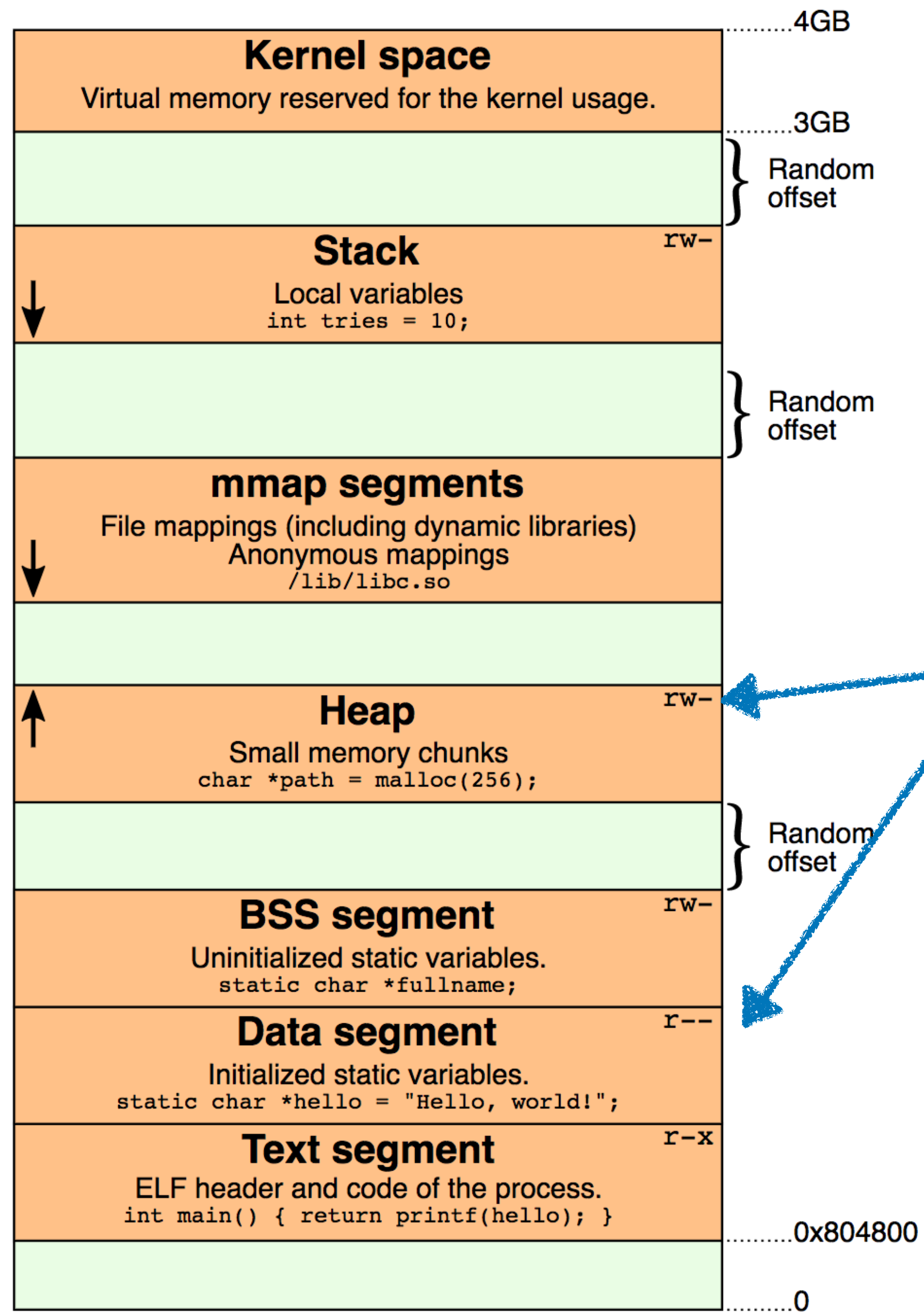
BSS: Uninitialized static
vars (globals)



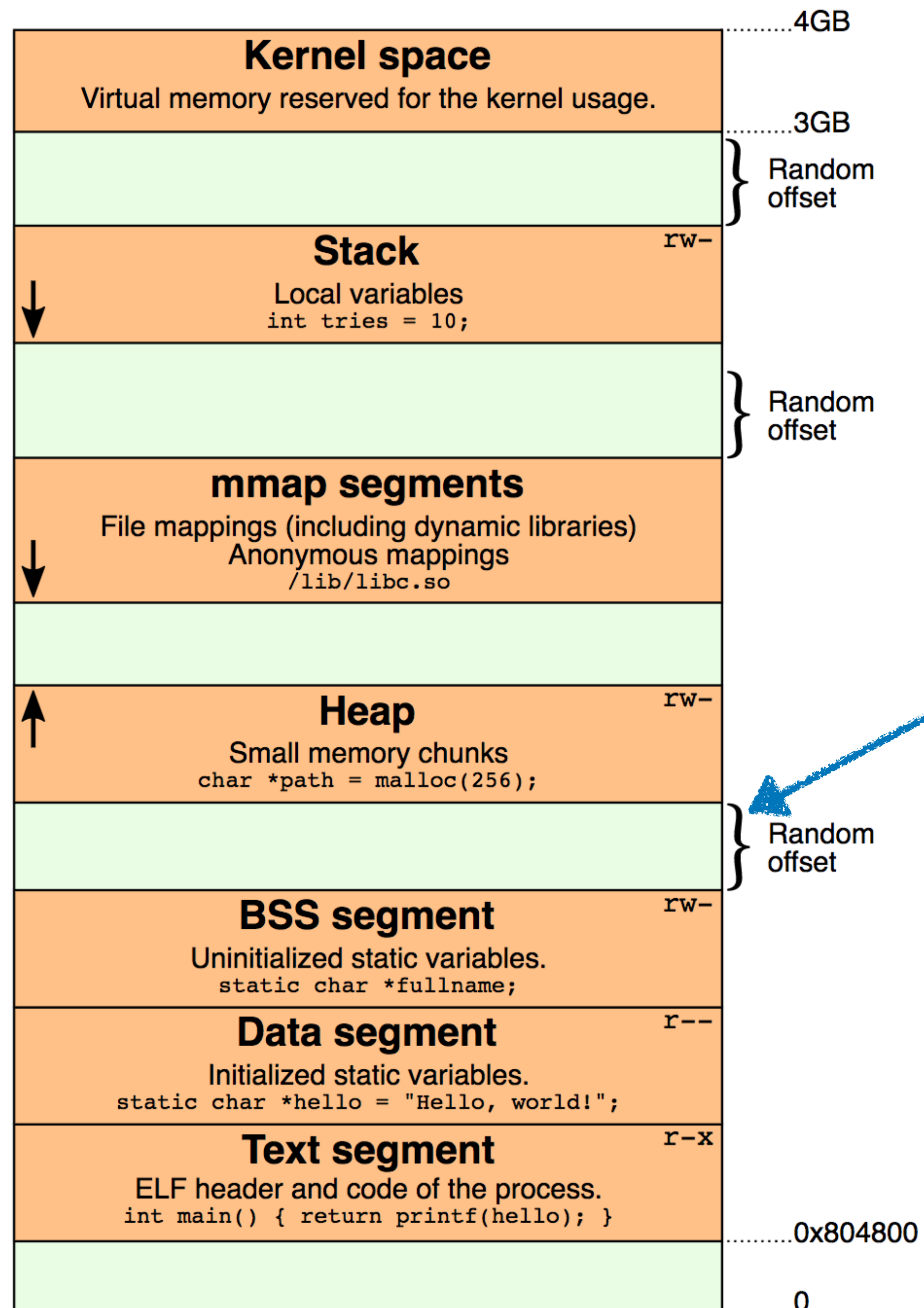
Data segment: initialized
statics—e.g., constant strings



Text segment: program code



Note the **permissions**



This **random offset** really security feature

What is the language, really?

Programs in our language are *straight-line blocks of arithmetic*:

<pre>x = read_int() y = 2*x+3 z = x + y*y return z+1</pre>		<pre>(let ([x (read)]) (let ([y (+ (* 2 x) 3)]) (let ([z (+ x (* y y))]) (+ z 1))))</pre>
--	---	---

Same program, ***just different syntax***

Compiling to x86

- We look at our source code, and we look at x86
- We ask: what are the differences?
 - First step: rewrite this in x86.
- Let's carefully walk through this:
 - What are the differences?

```
(program (let ([a 1]) a))
```

```
.globl _main
.extern _read_int64
.extern _print_int64
_main:
    pushq %rbp
    movq %rsp, %rbp
    addq $-16, %rsp
    movq $1, -8(%rbp)
    movq -8(%rbp), %rax
    movq %rax, %rdi
    call _print_int64
    movq $0, %rax
    movq %rbp, %rsp
    popq %rbp
    ret
```


Key differences of concern (Sec 2.3)

- (a) x86 arithmetic instructions update their “result” argument in place, this means we can’t implement $x := y + z$ (or similar) directly
- (b) in x86, our arguments must be **atomic** (variables or immediate values)
 - By contrast, in LVar, they can be arbitrary nested expressions!
 - We’ll need to do some flattening:
 - $(+ x (+ y z))$ can be written as $(\text{let } ([i0 (+ y z)]) (+ x i0))$
- (c) x86 has **explicit** control-flow: a block of instructions executing sequentially
 - LVar has an **implicit** control-flow, we need to make control-flow explicit
- (d) LVar can use any number of variables—only 16 registers in x86
 - Easy hack: use the stack instead
- (e) Variable names can be “shadowed,” but registers / stack can’t be in x86

The Big Idea

To get from source language to target language...

- Identify a set of *passes*, each of which takes *complex* features and desugars those features into more *primitive* (closer to the machine) features
- The interface between two passes constitutes an ***intermediate representation (IR)***
- *Questions*: can passes have the same input and output IR? Can passes ever be run in a loop (when might you want that)?

The Passes...

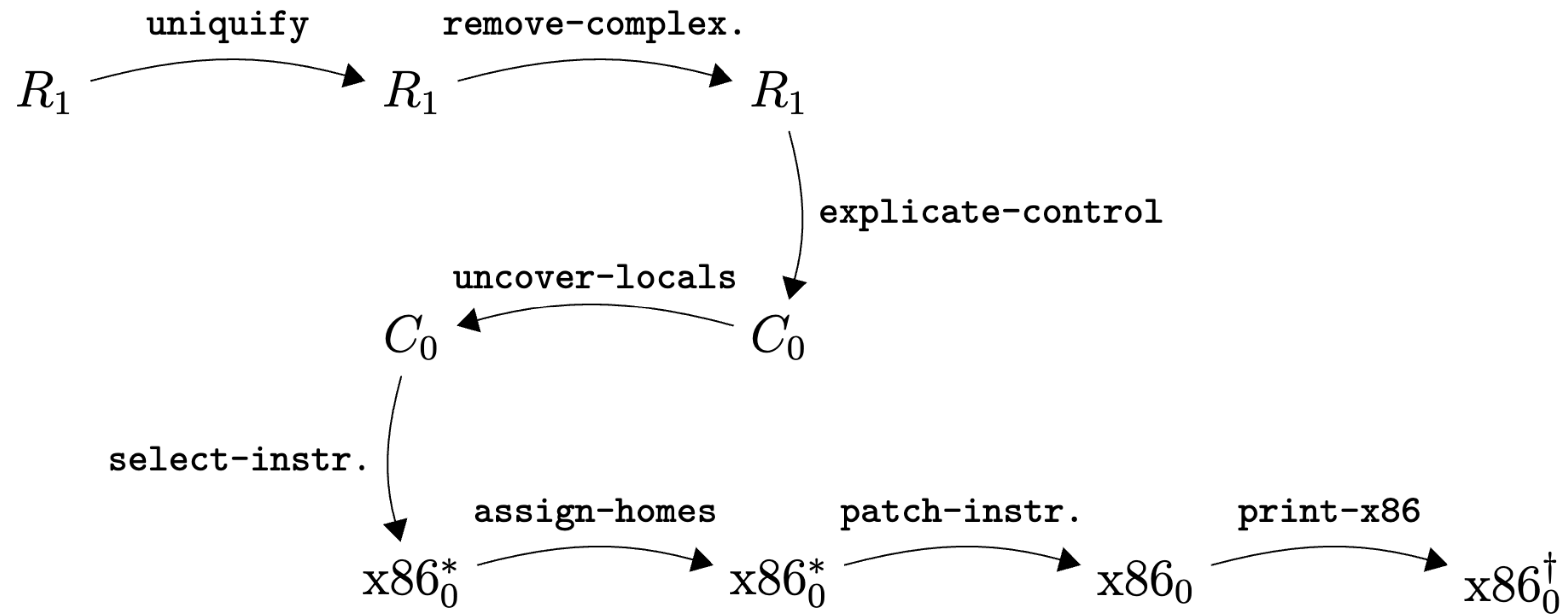


Figure 2.8: Overview of the passes for compiling R_1 .



Let's brainstorm how to transform a program into x86_64 gradually

- First insight: every operation in x86_64 needs to have **simple** arguments
 - Can't have $(+ (* x y) z)$, need $(\text{let } ([i (* x y)]) (+ i z))$
- Transform the program so that *complex* expressions are broken down into chains of simpler ones by introducing **administrative** bindings
- This process is called A-normal form conversion (or A-normalization)
 - The book calls it remove-complex-operand (-ator)

ANF

```
(let ([x (read)])  
  (let ([y (+ (* 2 x) 3)])  
    (let ([z (+ x (* y y))])  
      (+ z 1))))
```

```
(let ([x (read)])  
  (let ([i0 (* 2 x)])  
    (let ([y (+ i0 3)])  
      (let ([i1 (* y y)])  
        (let ([z (+ x i1)])  
          (let ([i2 (+ z 1)])  
            i2))))))
```


- Now we have a really ugly nested web of expressions.
- The key is to recognize that it is not an arbitrary tree—it is a **sequence**
- So in fact, we can just smush it together and linearize it
 - This will become more complex when we deal with branching control-flow
 - For now, things are easy—let's not make them complicated!
- The book calls this **explicate-control**
 - Also adds the explicit return statement

```
(let ([x (read)])
  (let ([i0 (* 2 x)])
    (let ([y (+ i0 3)])
      (let ([i1 (* y y)])
        (let ([z (+ x i1)])
          (let ([i2 (+ z 1)])
            i2))))))
```

Explicate control,
flatten control-flow

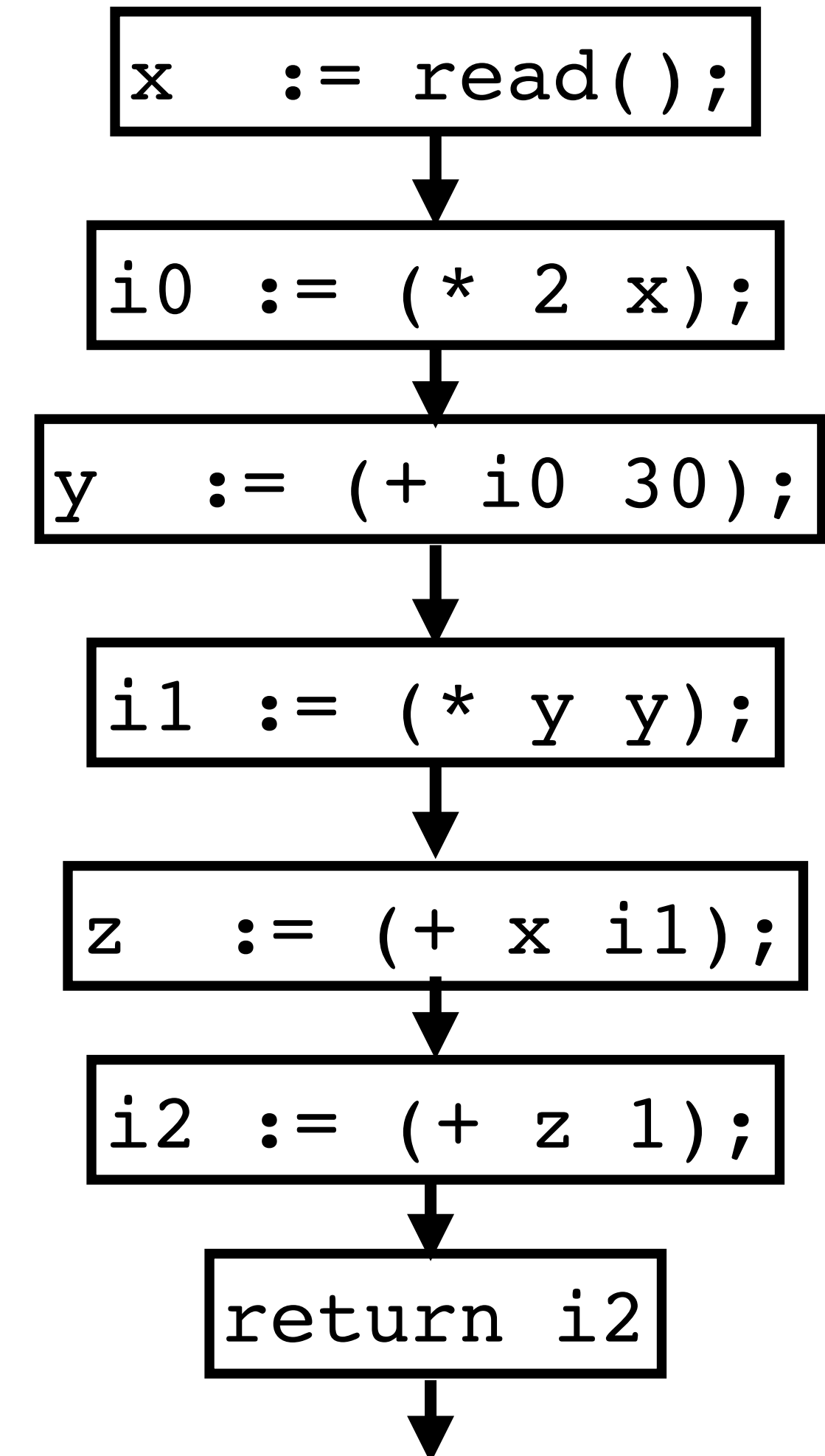


```
x := read();
i0 := (* 2 x);
y := (+ i0 30);
i1 := (* y y);
z := (+ x i1);
i2 := (+ z 1);
return i2
```

- In general, we will want to talk about a **control-flow graph**, to enable the potential for **branching** (and also *joining* after diverging)
- We can view a linear sequence as a kind of degenerate graph, where each line is a node and each node links to the next
- A straight line sequence of code is called a **basic block**

Basic Block

```
x := read();
i0 := (* 2 x);
y := (+ i0 30);
i1 := (* y y);
z := (+ x i1);
i2 := (+ z 1);
return i2
```

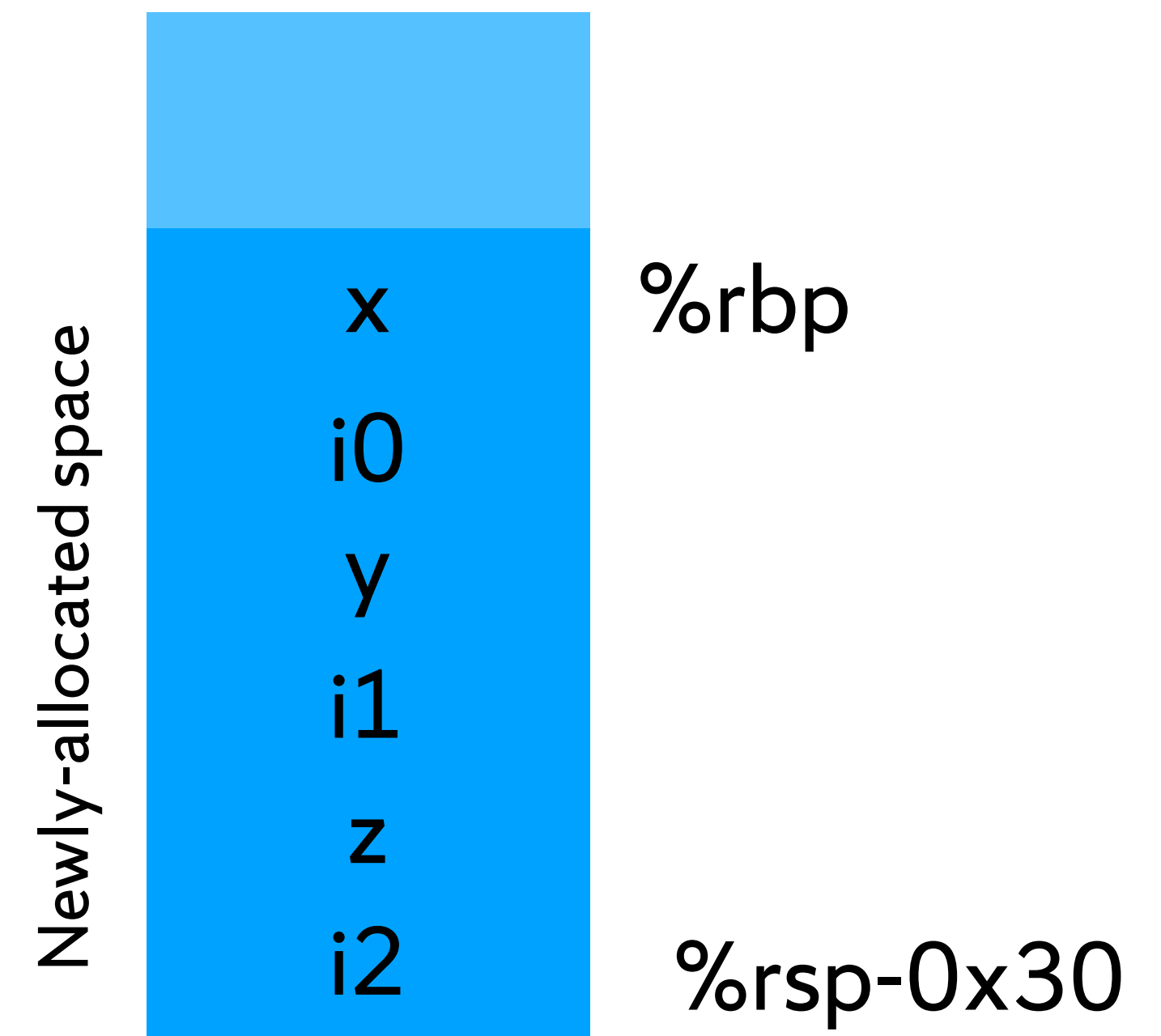


- Now that we have something like this, we're very close to assembly
- Need to allocate everything to a register and go
 - Won't work in general—we'll run out of registers
 - Let's make things dirt-simple—allocate space on the stack for each intermediate value
 - Left-hand-side of each assignment then implicitly writes to that stack location
- To do this, **collect up all local variables** and assemble a map of each local to a home on the stack, which we can then use to assign them...

```

x    := read( );      # -8( %rbp )
i0   := (* 2 x);      # -16( %rbp )
y    := (+ i0 30);    # -24( %rbp )
i1   := (* y y);      # -32( %rbp )
z    := (+ x i1);     # -40( %rbp )
i2   := (+ z 1);      # -48( %rbp )
return i2

```



- Now we'll do **instruction assignment**
 - Each top-level complex expression maps to (multiple) assembly instructions
 - Whenever a result gets left in a register (that should be in a variable), we move it
 - (Note: this project doesn't use *, I just include it here...)

```
x := read( );      # -8(%rbp)
i0 := (* 2 x);     # -16(%rbp)
y := (+ i0 30);    # -24(%rbp)
i1 := (* y y);     # -32(%rbp)
z := (+ x i1);     # -40(%rbp)
i2 := (+ z 1);     # -48(%rbp)
return i2
```

```
# x := read( );
callq read_int
movq %rax, -8(%rbp)
# i0 := (* 2 x);
movq $2, %rax
imulq -8(%rbp), %rax
movq %rax, -16(%rbp)
# y := (+ i0 30);
movq -16(%rbp), %rax
addq 30, %rax
movq %rax, -24(%rbp)
... # pop %rbp, reset stack...
# Leave ret value in %rax
ret
```



My Advice

- The book is excellent, and makes thoughtful design choices
- If you follow the book, it gives an exact recipe for how to do the project
- But you might figure out your own way of doing it that diverges a bit from the way the book does it (e.g., is simpler)—that is natural and expected
 - In fact, a key part of the class is experimenting with different design choices; consider reimplementing parts of projects different ways, what changes?
- ***I recommend starting from the book's approach, and I will roughly follow it in class—you can take any solution as long as you produce correct input/output combinations!***
 - Project starters **will** deviate from the book
 - There are multiple versions of the book, I wanted my own version :-)

- * The book breaks this conversion apart into a collection of tiny passes, “nano”-passes.
- * Each of these “nano” passes performs a specific job, transforming an input program to an output program

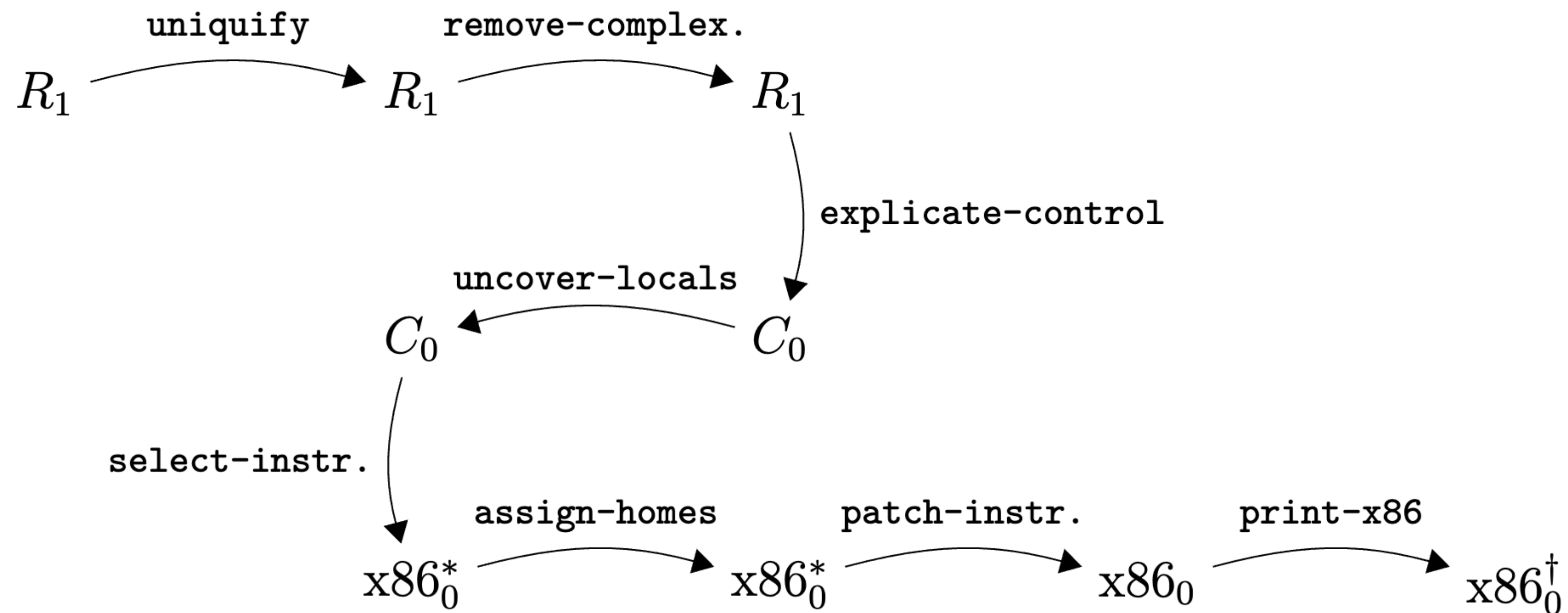


Figure 2.8: Overview of the passes for compiling R_1 .

- * Generally, a compiler pass will remove features
 - * Take a complex feature, implement it using multiple less-complex features
 - * This has pros/cons—some nuance here
- * Notice how as the page goes down, the abstraction level gets lower

Closeness to machine code

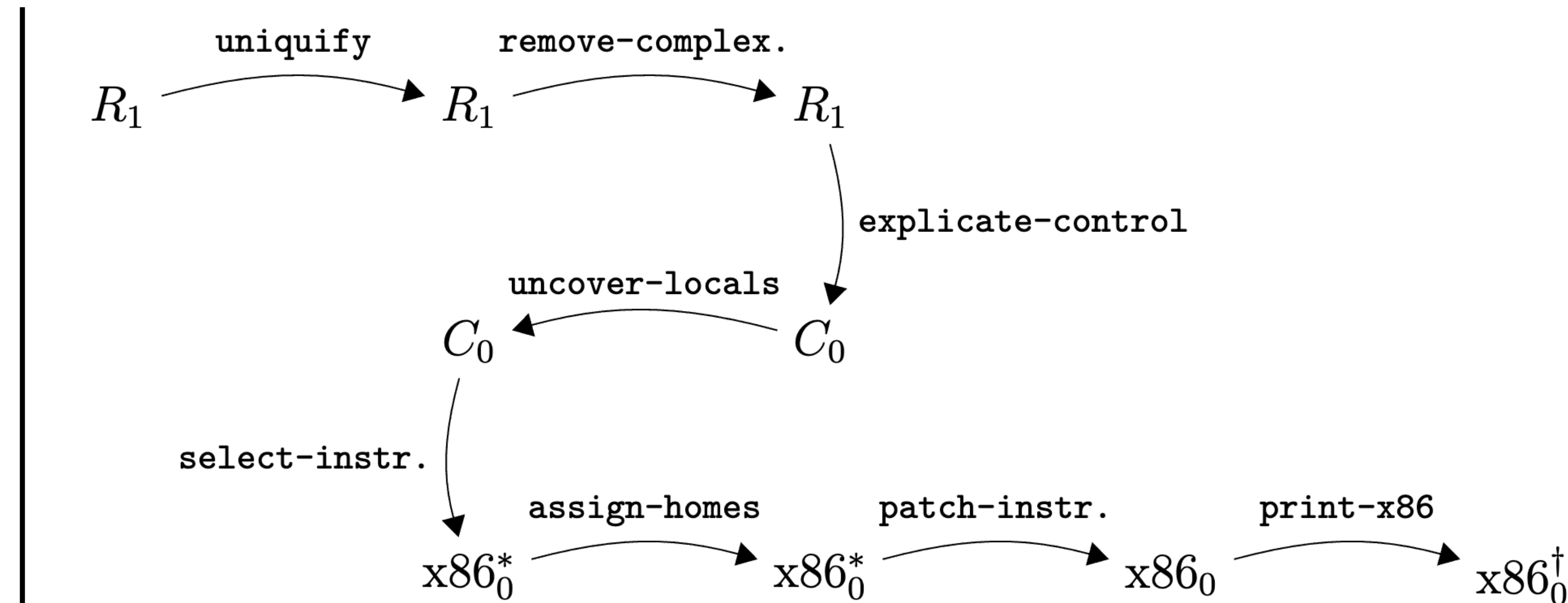
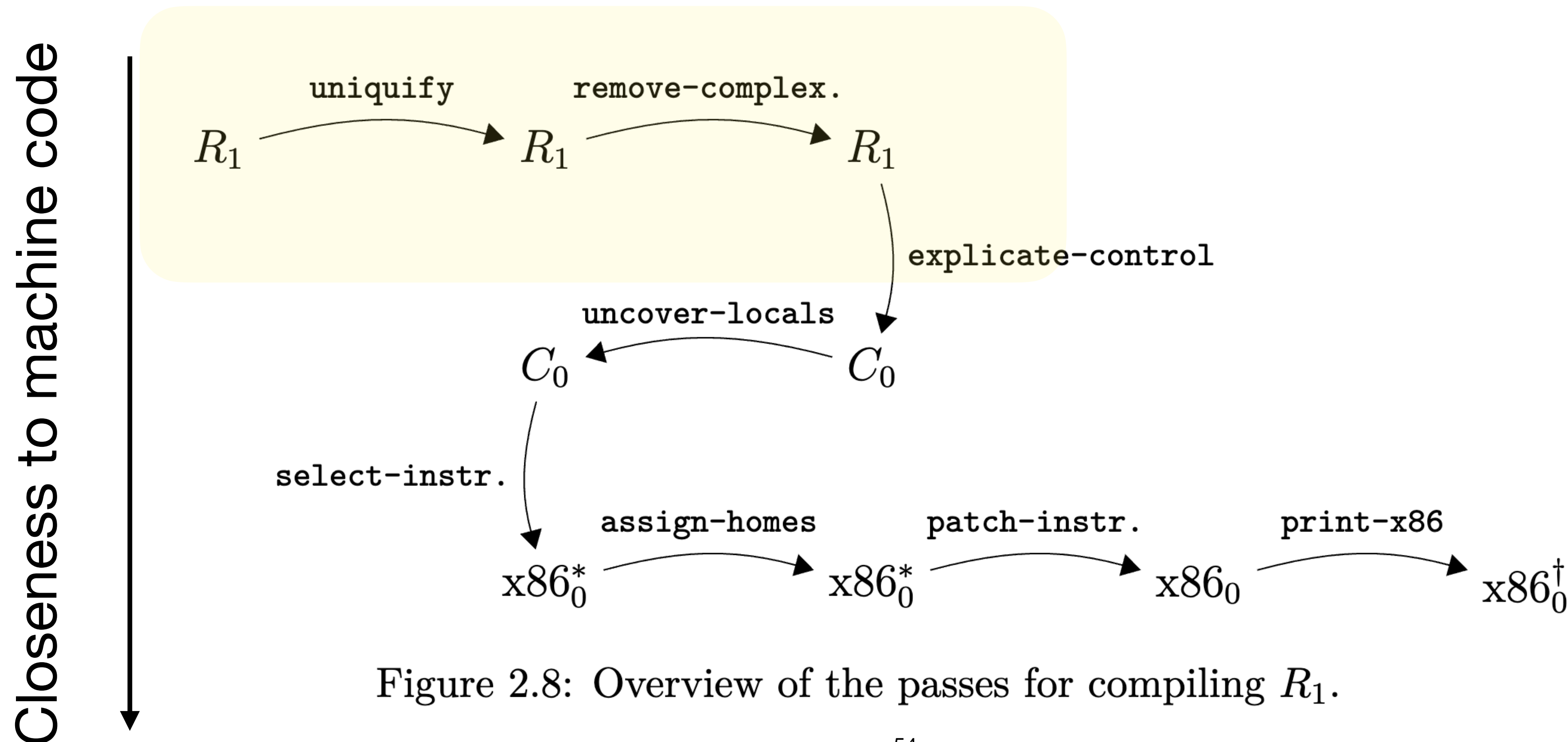


Figure 2.8: Overview of the passes for compiling R_1 .

- * These two passes operate on source-language code:
 - * `uniqueify` — alpha-renames variables to avoid name-clashes
 - * `remove-complex` — A-normalization to remove complex operations
 - * Introduces administrative bindings



- Explicate-control finally transforms the program into an IR:
 - IR tracks a **control-flow graph**, where program consists of **basic blocks**
 - Basic blocks—straight-line sequences of code—are ubiquitous
- uncover-locals finds out how much stack space we need

Closeness to machine code

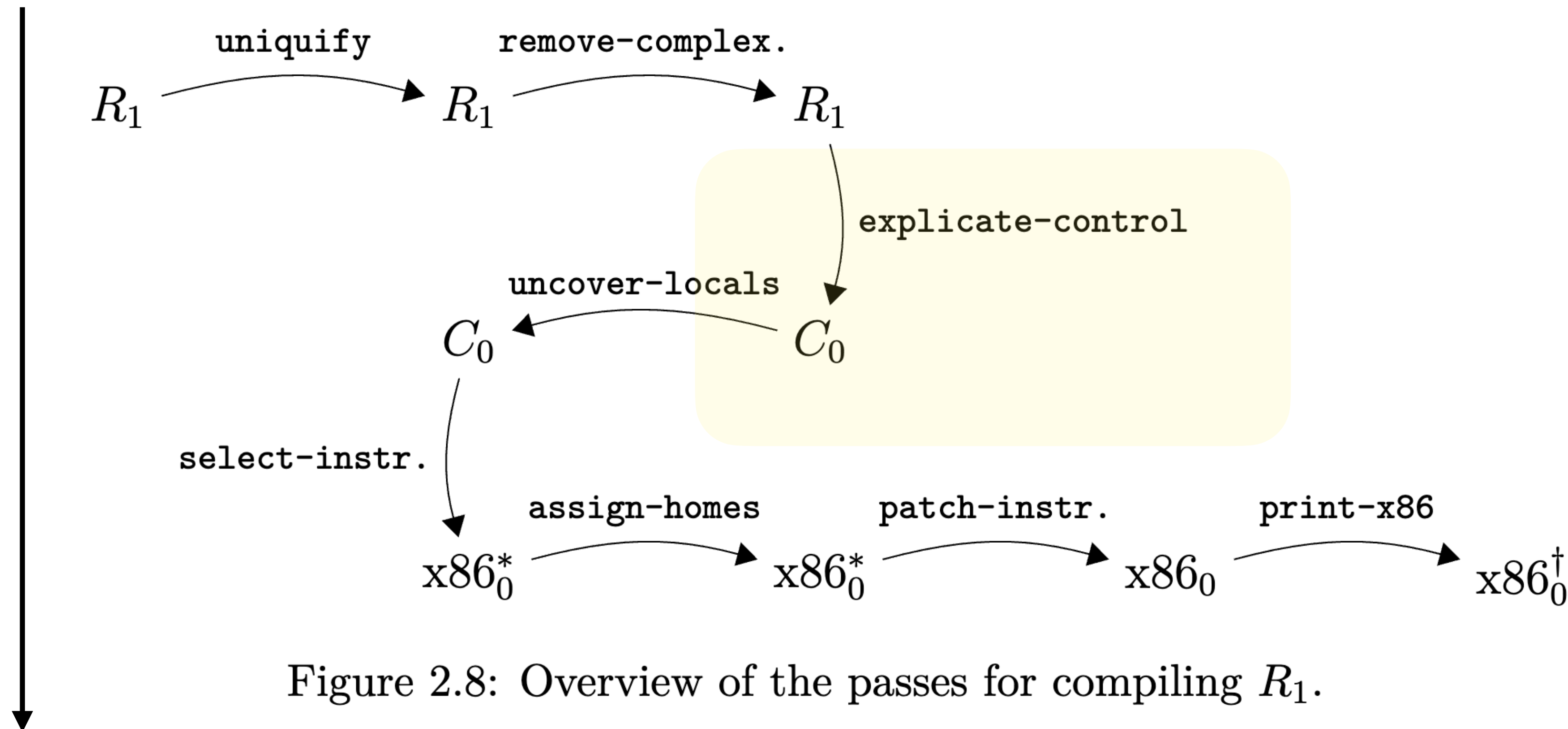


Figure 2.8: Overview of the passes for compiling R_1 .

- Finally, we're working on (increasingly-lower-level variants of) x86_64:
 - Generate specific instructions, while leaving variables abstract
 - Assigning “homes” to each variable
 - Patching instructions, to ensure we adhere to addressing conventions
 - Finally, print the x86 as a block of text

Closeness to machine code

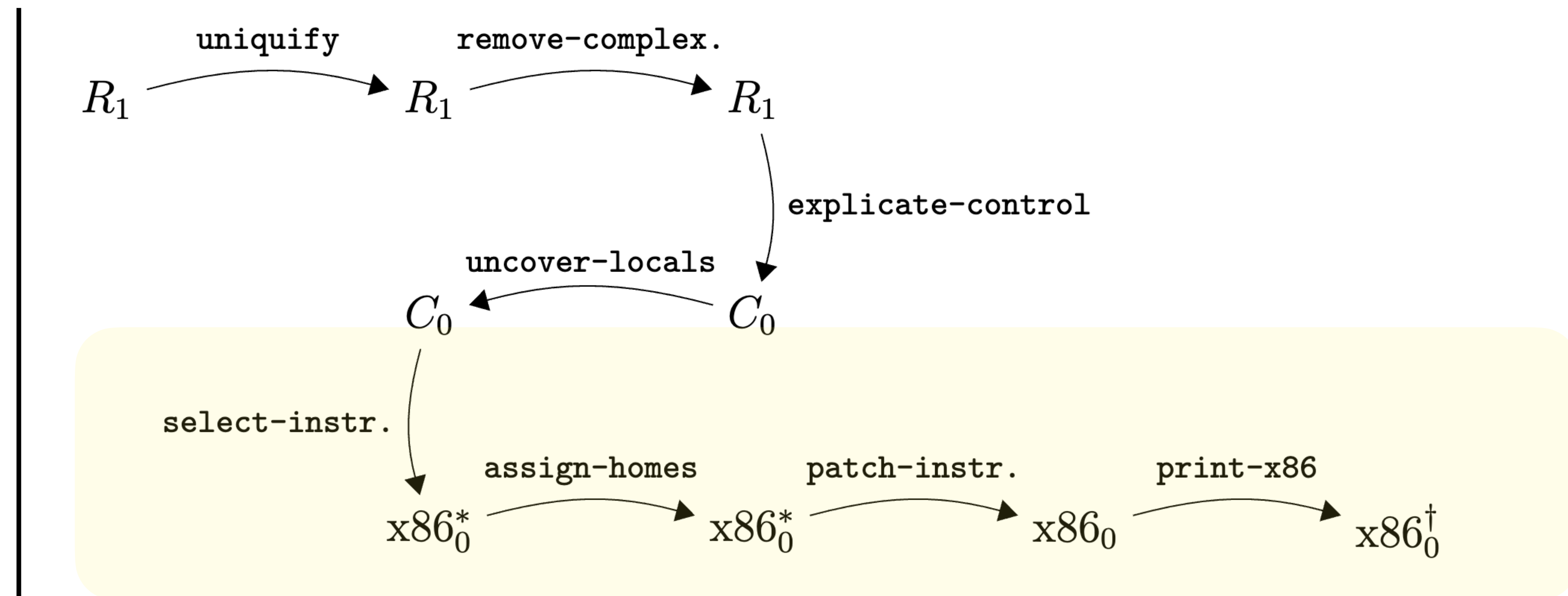


Figure 2.8: Overview of the passes for compiling R_1 .

In Detail...

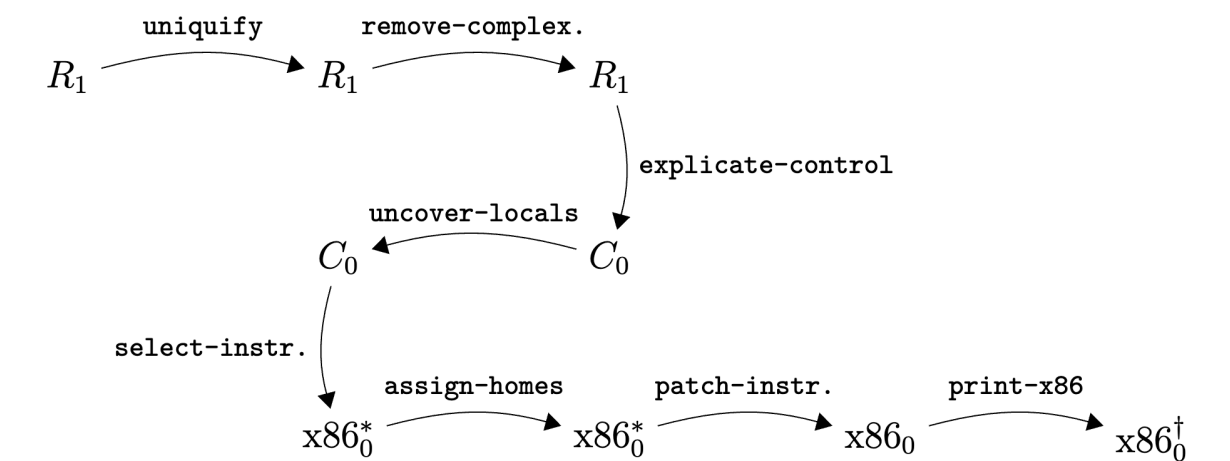


Figure 2.8: Overview of the passes for compiling R_1 .

1. **uniquify** → ensures every bound identifier is unique

Input: R_1 ? → Output: **unique-source-tree?** → Interp: **interpret-R1**

2. **anf-convert** → ANF conversion (introduce temps to flatten nested ops)

Input: **unique-source-tree?** → Output: **anf-program?** → Interp: **interpret-anf**

3. **explicate-control** → convert ANF to C0 (a statement/block flavor)

Input: **anf-program?** → Output: **c0-program?** → Interp: **interpret-c0**

4. **uncover-locals** → collect local variables (stack frame sizing later)

Input: **c0-program?** → Output: **locals-program?** → Interp: **interpret-c0**

5. **select-instructions** → map C0 statements to pseudo-x86 instructions over vars

Input: **locals-program?** → Output: **instr-program?** → Interp: **interpret-instr**

6. **assign-homes** → assign each var to a stack slot (**deref (reg rbp) offset**)

Input: **instr-program?** → Output: **homes-assigned-program?** → Interp: **interpret-instr**

7. **patch-instructions** → break illegal memory→memory **movq** into two moves via **%rax**

Input: **homes-assigned-program?** → Output: **patched-program?** → Interp: **interpret-instr**

8. **prelude-and-conclusion** → function prologue/epilogue; call **print_int64** on **%rax**

Input: **patched-program?** → Output: **x86-64?** → Interp: **interpret-instr**

9. **dump-x86-64** → render to assembler text (string)

Input: **x86-64?** → Output: **string?** → executed natively...

Uniqueify (“Alphatization”)

In Racket / Python / ... we can **shadow names**. This is problematic for a compiler: let’s say you decide you want to rewrite `x` to be something else: now, you can’t replace `x` without disambiguating!

We **alpha-convert** unique instances of `x` to (gensym)’d names (“alphatization”)

```
'(program
  (let ((x (read)))
    (let ((y (let ((x (read))) (+ x (+ x x)))))
      (+ (- (+ x y)) x))))
```

```
-> unique
'(program
  ()
  (let ((x (read)))
    (let ((y (let ((x1488 (read))) (+ x1488 (+ x1488 x1488)))))
      (+ (- (+ x y)) x))))
```

Converting to “simple” arguments

Most languages (Racket, C/C++, Java, JS) allow “complex” (i.e., nested) arguments to functions and other forms such as if, etc.

However, this is problematic for a compiler: the computer runs on a clock, and there’s no way to ensure that a complex expression can complete all in one clock cycle...

```
(let ([f (g (+ 3 (x)))]  
      [h (+ 5 (* (g 2) 4))])  
  (f h))
```

For example, above we have a complex expression. It is possible to rewrite it in a simpler form...

The diagram illustrates the transformation of a nested `let` expression into a flattened sequence of `let` expressions using administrative bindings. On the left, a nested `let` expression is shown: `(let ([f (g (+ 3 (x)))] [h (+ 5 (* (g 2) 4))]) (f h))`. An arrow points from this expression to the right, where the flattened version is shown. The flattened version consists of a sequence of `let` expressions: `(let ([int0 (x)]) (let ([int1 (+ 3 int0)]) (let ([int2 (g int1)]) (let ([f int2]) (let ([int3 (g 2)]) (let ([int4 (* int3 4)]) (let ([int5 (+ 5 int4)]) (let ([h int5]) (f h))))))))))`. The arrow indicates the mapping from the original nested structure to this flattened sequence.

Basic idea: create an **administrative (intermediate) binding** for *each subexpression*. Use the administrative names as arguments to flatten complex expressions into sequences of “simple” expressions.

A simple expression is a function applied to atoms. An atom is either a variable or a constant.


```

      (let ([int0 (x)])
        (let ([int1 (+ 3 int0)])
          (let ([int2 (g int1)])
            (let ([f int2])
              (let ([int3 (g 2)])
                (let ([int4 (* int3 4)])
                  (let ([int5 (+ 5 int4)])
                    (let ([h int5])
                      (f h))))))))))

(let ([f (g (+ 3 (x)))]
      [h (+ 5 (* (g 2) 4))])
  (f h))

```



We will talk about this more next lecture, it is tricky and relates to a well-known compilers topic (Static Single Assignment, SSA)!

explicate-control

explicate-control is the pass that takes us from “expressions” to “blocks.” In an expression, there is nested structure based on lets, etc.

The output of explicate-control

```
-> unique
'(program
  ()
  (let ((x (read)))
    (let ((y (let ((x1488 (read))) (+ x1488 (+ x1488 x1488)))))
      (+ (- (+ x y)) x))))
```

uncover-locals

I wrote this pass for you, no need to implement it—it collects up the local variables for use in subsequent passes (e.g., when they get assigned stack locations)

patch-instructions

This pass rewrites any instructions which generate memory/memory operands

Idea: identify any operations where both arguments are in memory, and add extra **movq** instructions

prelude-and-conclusion

I have scaffolded most of this pass, along with dump-x86-64